Claude Code assessment preparation

Step 1 of 2

Updated tasks, ran a command, loaded tools

AICC102 Expense Management API — Complete Remaining Assessment
You are the lead software engineer responsible for completing my AICC102 Claude Code assessment project end to end.
My current working directory is the assessment project folder. First, verify the actual path. The intended project directory is:
`C:\Users\Laxmi Bandaru\claude-code-workshop-assessment`
The demo project is located at:
`C:\Users\Laxmi Bandaru\claude-code-workshop`
The demo project is STRICTLY READ-ONLY. Never modify, create, delete, move, or run build/install/Git commands inside it.
1. Main objective
I have already completed work through approximately Phase 13 of my assessment playbook, including creating hooks. Do not assume every earlier step is complete or correct.
Your task is to:

1. Inspect the current project and identify exactly what is already implemented.
2. Preserve all correct existing work and avoid overwriting completed files.
3. Complete missing or incorrect Claude Code configuration and assessment features.
4. Finish the ASP.NET Core backend and React frontend as a functional application.
5. Connect the frontend to the backend and SQLite database.
6. Use the project's rules, skills, subagents, hooks, MCP integration, and Cognine plugin appropriately during actual implementation and verification.
7. Run the available automated tests and builds.
8. Produce honest assessment evidence, a working README, a demo script, and assessor Q&A.

Do not merely create configuration files that look correct. Demonstrate that the configured features actually work.
2. Mandatory safety rules

- Work only inside the assessment repository, except for explicitly permitted local Git worktree operations.
- Never access or modify the demo project beyond read-only inspection that has already been authorized.

- Do not discard existing changes, reset branches, force-push, or clean the Git working tree.
- Do not overwrite existing files without inspecting their contents first.
- Do not delete a database, migrations, user data, or Git history.
- Do not weaken, skip, or remove tests to obtain a passing result.
- Never invent command output, test counts, plugin availability, MCP connectivity, or evidence.
- Never put secrets or credentials in source control.
- Use explicit C# types instead of `var`, following the project rules.
- Follow the existing architecture, `CLAUDE.md`, path-scoped rules, skills, and security hooks.
- Keep API controllers thin; implement business rules in services.
- Use typed React/TypeScript code without `any`.
- Keep all HTTP requests in the frontend API layer.
- Do not introduce dependencies without explaining why and obtaining approval where the project rules require it.
- Before destructive, potentially irreversible, or broad changes, explain the impact and wait for my explicit approval.
- Preserve existing `.claude/settings.json`, enabled plugins, hook configuration, `.mcp.json`, Git changes, and working features unless a verified issue requires a change.
- Do not claim a hook or subagent worked unless its execution is visible in the actual results or logs.

3. Stage A — Read-only project audit
   Start in Plan Mode.
   Before changing anything, inspect:

- Current working directory and Git branch/status.
- `CLAUDE.md`.
- `docs/requirements.md`.
- `docs/architecture.md`.
- `docs/plugin-evaluation.md`.
- `docs/reuse-assessment.md`.
- `.claude/settings.json`.
- `.mcp.json`, if present.
- All files under `.claude/rules/`, `.claude/skills/`, `.claude/agents/`, and `.claude/hooks/`.
- Existing backend and frontend source files.
- Existing migrations, tests, `.http` samples, and assessment evidence.
- The current `README.md`, if present.

Use file listings and Git status to establish what actually exists. Do not infer completion from documentation alone.
Create a status matrix with these columns:

```
Requirement | Expected artifact/behaviour | Actual evidence |
```

`Status | Remaining work`
Audit at least the following:

- Requirements and architecture documentation.
- Backend solution and API scaffolding.
- SQLite database, entities, EF Core configuration, migrations, and development seed data.
- `CLAUDE.md` and all six rules, including path-scoped frontmatter.
- Reusable `aspnet-endpoint-workflow` skill.
- Project-specific `expense-reporting` skill.
- `api-implementer` and `qa-reviewer` subagents.
- PreToolUse and PostToolUse hooks.
- Hook test results and logs.
- SQLite MCP server configuration and read-only restrictions.
- Cognine Marketplace and React plugin availability.
- Categories, expenses, budgets, summaries, and reports endpoints.
- React pages, typed API client, validation, and error handling.
- Backend and frontend automated tests.
- Git commits, verification reports, and demo evidence.

Distinguish `PASS`, `PARTIAL`, `MISSING`, `FAIL`, and `BLOCKED`.
For every PASS, identify concrete evidence. For example, a file proves existence, but a successful hook test proves execution.
Do not modify files during this stage.
4. Stage B — Plan the remaining work
After the audit, prepare a prioritised remediation plan.
Use the existing AICC102 playbook as the primary specification. Preserve its fixed decisions:

- ASP.NET Core Web API using .NET 10 and controllers.
- API project: `backend/ExpenseManagement.Api`.
- Test project: `backend/ExpenseManagement.Api.Tests`.
- API URL: `http://localhost:5200`.
- React + TypeScript + Vite frontend under `frontend/`.
- Frontend URL: `http://localhost:5173`.
- SQLite database: `expense-management.db`.
- EF Core migrations.
- xUnit backend tests.
- Vitest and React Testing Library frontend tests.
- RFC 7807 ProblemDetails for API errors.
- Hard budgets reject an expense that exceeds the limit with HTTP 422.
- Soft budgets accept the expense and return a warning.
- Monthly budget statuses: OnTrack below 80%, Warning from 80% through 100%, Exceeded above 100%.
- Explicit C# types, layered architecture, and least-privilege MCP access.

Identify any contradictions between existing implementation, requirements, and architecture. Do not silently change the agreed API contract.

Present the plan with dependencies, affected files, risks, and validation commands.
Wait for my approval before beginning broad implementation. After approval, execute the work in small, reviewable stages. Do not ask for approval after every harmless file edit; follow the approved plan and the project's safety rules.
5. Stage C — Complete Claude Code configuration
Complete missing items without recreating working files.
A. CLAUDE.md
Ensure it accurately documents:

- Project purpose and technology stack.
- Backend/frontend run and test commands.
- API and frontend ports.
- SQLite connection and migration workflow.
- Architecture and folder structure.
- Business validation and budget rules.
- All existing rules, skills, subagents, hooks, MCP tools, and plugin usage.
- Definition of done.

Mark an item as implemented only if it exists and has been verified. Remove obsolete `planned` labels only after confirming the feature works.
B. Path-scoped rules
Verify that the project contains the six intended rule files:

- `dotnet-backend.md`
- `api-design.md`
- `data-and-migrations.md`
- `react-typescript.md`
- `testing.md`
- `security.md`

The first five must have correct YAML `paths:` frontmatter. `security.md` must apply globally.
Check the path patterns against real files. Verify rule-loading behaviour using the installed Claude Code version and documented commands. If path-scoped rules do not load as expected, diagnose the root cause rather than claiming success.
C. Skills
Verify and complete:

1. `.claude/skills/aspnet-endpoint-workflow/SKILL.md`
2. `.claude/skills/aspnet-endpoint-workflow/checklist.md`
3. `.claude/skills/expense-reporting/SKILL.md`

The first skill must be reusable across ASP.NET Core Web API + EF Core projects and must not depend on this application's expense domain.
The second skill must encode the application's month boundaries, decimal precision, SQLite aggregation limitations, summary structure, budget status rules, CSV formatting, MCP cross-check procedure, and required tests.

Use each skill during its relevant implementation work. Do not merely list it in `CLAUDE.md`.
D. Subagents
Verify and complete:

- `.claude/agents/api-implementer.md`
- `.claude/agents/qa-reviewer.md`

`api-implementer` owns backend implementation and may edit backend files, but must not edit frontend files or Claude Code configuration.
`qa-reviewer` must be an independent reviewer without Edit or Write tools. It must not modify code.
Use `api-implementer` for appropriate backend tasks and `qa-reviewer` after each meaningful backend feature. Capture actual findings and results.
E. Hooks
Verify the existing pre-hook and post-hook before changing them.
Expected files:

- `.claude/hooks/pre-guard.cs`
- `.claude/hooks/post-verify.cs`

The pre-hook should block the specified destructive commands and protected file changes. The post-hook should build-check changed C# files and type-check changed TypeScript files.
Check:

- Hook registration and matchers in `.claude/settings.json`.
- Correct commands, quoting, timeout values, and file paths.
- Exit-code behaviour.
- Logging to `.claude/logs/hooks.log`.
- Safe behaviour for malformed input and log failures.
- Compatibility with the installed Claude Code and .NET versions.

If hook tests have not been run, complete them safely with controlled test payloads after reviewing the implementation. Do not execute genuinely destructive commands or delete real data to demonstrate blocking. Use safe test inputs that the hook can intercept.
Fix and retest defects. Preserve all existing plugin settings.
F. MCP
Verify the SQLite MCP server using the installed Claude Code tools and the actual `.mcp.json`.
The intended server is `expense-db`, restricted to read-only database inspection.

- Prefer a snapshot database under `.claude/mcp/expense-snapshot.db`.
- Never expose unrestricted write access to the development database.
- Inspect the actual tools exposed by the installed server before configuring permissions.
- Deny every identified write-capable MCP tool using the supported permission syntax.

- Give the `qa-reviewer` only the read-only MCP tools it needs, if supported.
- Verify `/mcp` reports a connected server and usable read-only tools.
- Run actual schema and SELECT checks through MCP.
- Demonstrate that a write attempt is denied by the configured permissions.
- Document how to refresh the snapshot safely.

Do not assume the package name, tool names, permission syntax, or connectivity. Check the installed version and real output. If the server cannot connect, report the error and resolve it safely.

G. Cognine Marketplace plugin

Inspect the real Marketplace configuration and available plugin commands.

- Preserve existing plugin settings.
- Confirm whether `react-coding-standards-best-practices` is available and enabled.
- Confirm the actual command names rather than assuming them.
- Run available architecture, secure-coding, and accessibility audits on the frontend.
- Save before-and-after results in the required evidence files.
- Fix genuine Mandatory/High findings without breaking behaviour.
- Run frontend tests and build after fixes.
- If the plugin or a command is unavailable, document the actual result and the next step. Never fabricate plugin output.

After modifying startup-loaded configuration, tell me when a new Claude Code session is required. Do not claim new settings are loaded until verified.

6. Stage D — Complete the backend

Follow `docs/requirements.md`, `docs/architecture.md`, and the project rules.

First inspect the current code. Implement only missing or defective functionality.

Data and persistence

Ensure that the application has:

- Category, Expense, Budget, and BudgetEnforcement domain types.
- AppDbContext with Fluent API configuration.
- Decimal precision of 18,2.
- DateOnly for expense dates.
- Unique category names using case-insensitive comparison.
- A unique budget index for category, year, and month.
- Restrict foreign-key delete behaviour.
- Correct lengths, validation, and enum storage.
- EF Core migrations created through `dotnet ef`.
- SQLite connection configured for `expense-management.db`.
- Development-only seed data that never runs in tests.
- Tests using isolated in-memory SQLite, never the development database.

Do not apply migrations, overwrite a database, or change existing migration history without inspecting the current state and confirming safety.
Shared API foundation
Ensure the API has:

- A generic `ServiceResult<T>` or the established equivalent.
- Thin controllers.
- Business logic in services.
- Dependency injection.
- RFC 7807 ProblemDetails for all non-success responses.
- A generic 500 response that does not leak exception details.
- Consistent validation and error mapping.
- CORS for the local Vite frontend.
- OpenAPI JSON and a usable API documentation UI.
- An HTTP request sample file with a base URL of `http://localhost:5200`.

Categories
Implement or verify:

- `GET /api/categories`
- `GET /api/categories/{id}`
- `POST /api/categories`
- `PUT /api/categories/{id}`
- `DELETE /api/categories/{id}`

Enforce unique category names and return 409 for duplicates. Return 409 when deleting a category that has expenses, 404 for a missing category, and 204 for successful deletion.
Expenses
Implement or verify CRUD and the list endpoint.
Validation:

- Title: required, 1–100 characters.
- Amount: greater than zero, no more than 1,000,000, maximum two decimal places.
- Date: required and not in the future.
- Category: must exist.
- Notes: optional, maximum 500 characters.

The list endpoint must support date range, category, amount range, case-insensitive search, sorting, and paging.
Use SQL filtering where appropriate. Because SQLite has limitations around decimal aggregation and sorting, follow the agreed rule to materialise the filtered data and perform the relevant decimal calculations in C#.
Budgets
Implement or verify budget CRUD and budget status.

- Limit must be greater than zero.
- Month must be 1–12.
- Year must be 2000–2100.

- One budget per category per month.
- Hard budgets reject overspending with 422.
- Soft budgets accept overspending and return a warning.
- Expense create and update must both enforce budget rules.
- Updating an expense must exclude the existing expense from the calculation and handle changes to category and month correctly.

Test exact-limit, over-limit, warning threshold, and exceeded threshold cases.
Summaries and reports
Implement or verify:

- `GET /api/summaries/monthly?year=&month=`
- `GET /api/reports/expenses?from=&to=&categoryId=&format=json|csv`

Monthly summaries must include totals, counts, category totals and percentages, and budget status. Empty months must return zeros rather than 404.
JSON reports must include the requested date range, generation time, total amount, expense count, category breakdown, and expenses.
CSV must use the agreed header, ISO dates, invariant culture, correct quoting/escaping, `text/csv`, and the specified filename pattern.
Follow the `expense-reporting` skill for calculations and verification.
Backend testing
Use xUnit and the project's established testing approach.
Cover success, validation, missing resources, conflicts, hard/soft budget behaviour, date boundaries, sorting, paging, summaries, rounding, CSV escaping, and error handling.
Do not use the real development database in tests. Do not weaken or skip tests.
After each feature:

1. Delegate implementation appropriately to `api-implementer`.
2. Run build and relevant tests.
3. Invoke `qa-reviewer` for independent review.
4. Fix verified defects.
5. Record real results and commit at the agreed milestone.
6. Stage E — Complete the frontend
   Work only in `frontend/` for frontend implementation, except for approved assessment evidence files.
   Follow the backend API contract and `.claude/rules/react-typescript.md`.
   Foundation
   Ensure the frontend has:

- Strict TypeScript.
- Typed DTOs matching the actual backend contract.
- A typed API client that parses ProblemDetails into a structured `ApiError`.
- API modules for categories, expenses, budgets, summaries, and reports.
- Reusable loading, empty, error, and confirmation components.

- Simple navigation for Expenses, Categories, Budgets, and Summary & Reports.
- A Vite `/api` proxy targeting `http://localhost:5200`.

Expenses page
Implement:

- Paged expense table.
- Date, category, amount, search, and sort filters.
- Add/edit form with matching client-side validation.
- Delete confirmation.
- Field-level validation messages.
- Clear 422 hard-budget errors.
- Soft-budget warnings after successful saves.
- Loading, empty, and error states.

Categories page
Implement list, add, rename, and delete. Display server validation and the 409 message when a category is in use.
Budgets page
Implement month selection, budget CRUD, Hard/Soft selection, and budget status with a progress bar and visible text labels. Never communicate status through colour alone.
Summary and Reports page
Implement monthly totals, category percentages, budget status, JSON report viewing, and CSV download. Display API validation errors clearly.
Frontend testing
Use Vitest and React Testing Library with mocked fetch calls. Cover form validation, expense creation, budget errors/warnings, filter query parameters, empty/error states, summary rendering, and CSV download.
Run `npm test` and `npm run build`. Report the actual counts and results.
If the API is not running, use mocked tests and clearly distinguish them from live integration verification.
8. Stage F — Integration and contract alignment
After both implementations are complete:

1. Start the backend and frontend in separate terminals.
2. Inspect the actual OpenAPI document.
3. Compare every frontend DTO, endpoint, query parameter, enum, and error response with the backend contract.
4. Fix frontend contract mismatches in the frontend.
5. Report real backend defects rather than silently changing the contract.
6. Test all pages against the live API.
7. Verify persistence by restarting the API and confirming that test records still exist.
8. Verify monthly summary totals against a read-only SELECT query through the MCP snapshot.
9. Create only uniquely identifiable E2E test records, and delete only those records by their IDs after the tests.

10. Never delete existing user or seed data to simplify testing.

Run:

- `dotnet build backend`
- `dotnet test backend`
- `npm test` from `frontend/`
- `npm run build` from `frontend/`

Run additional coverage or lint checks only when the required tools are installed and configured. Report missing tools honestly.
9. Stage G — Complete the assessment evidence
Create or update the following files based on real evidence:

- `docs/verification.md`
- `docs/claude-code-evidence.md`
- `docs/evidence/plugin-audit-before.md`
- `docs/evidence/plugin-audit-after.md`
- `docs/demo-script.md`
- `README.md`

Do not replace existing evidence without inspecting it first.
Verification report
Map every FR-xx requirement to the actual request, expected status, actual status, PASS/FAIL, and supporting test or command.
Include categories, expenses, validation, filtering, sorting, paging, budgets, monthly summaries, JSON/CSV reports, ProblemDetails, and persistence.
Claude Code evidence
For each mandatory feature, document:

- File/configuration path.
- Purpose.
- Actual phase or prompt where it was used.
- Relevant Git commit hash.
- Real evidence, such as hook log entries, test output, MCP query results, subagent review, or plugin audit reports.

Include CLAUDE.md, each rule, both skills, both subagents, MCP, both hooks, and the Cognine plugin.
Mark any feature that was created but never demonstrated as `NOT DEMONSTRATED`. Do not invent evidence to make the checklist appear complete.
README and demo
Provide prerequisites, setup instructions, run commands, test commands, endpoint overview, and links to the project documentation.
Create a 10-minute demo script with exact commands or prompts, expected results, and short explanations suitable for an assessor.
Include concise answers to likely questions about:

- CLAUDE.md vs rules vs skills.
- Skills vs subagents.
- Path-scoped rules.

- PreToolUse vs PostToolUse hooks.
- Why the reviewer cannot edit files.
- MCP read-only permissions.
- Cognine plugin verification.
- SQLite decimal limitations.
- Test database isolation.
- Hard vs Soft budgets and HTTP 422.

10. Stage H — Git and final verification
    Preserve the current branch and existing work.
    At the beginning, identify uncommitted changes. Do not commit unrelated user changes.
    After the approved implementation milestones, make small, meaningful commits with clear messages. Do not force-push or rewrite history.
    Before the final commit:

- Review `git diff` and `git status`.
- Check that no secrets, databases, build outputs, or unintended files are tracked.
- Confirm no files were modified outside the assessment project, except explicitly authorised worktree operations.
- Run the final backend and frontend checks.
- Update documentation to match the implementation.
- List all remaining limitations.

11. How to work with me
    Use this sequence:

    1. Audit first. No edits in Stage A.
    2. Show me the status matrix and remediation plan.
    3. Wait for approval before broad implementation.
    4. Complete work in small phases and validate each phase before proceeding.
    5. Use the configured skills, subagents, hooks, MCP, and plugin where relevant.
    6. Show actual test results and evidence, not assumptions.
    7. Ask for my approval when a decision is destructive, irreversible, or materially changes the agreed contract.
    8. Do not ask me to repeat information already present in the project documentation.
    9. If something is blocked by permissions, missing software, unavailable plugin commands, or a failed MCP server, explain the exact blocker and the smallest next step.
    10. Finish with a concise table showing each requirement, status, and evidence.

Begin now with Stage A only. Do not edit, install, migrate, seed, delete, or commit anything during the initial audit.

Show more

AICC102 Expense Management API — Complete Remaining Assessment
You are the lead software engineer responsible for completing my AICC102 Claude Code assessment project end to end.
My current working directory is the assessment project folder. First, verify the actual path. The intended project directory is:
`C:\Users\Laxmi Bandaru\claude-code-workshop-assessment`
The demo project is located at:
`C:\Users\Laxmi Bandaru\claude-code-workshop`
The demo project is STRICTLY READ-ONLY. Never modify, create, delete, move, or run build/install/Git commands inside it.
1. Main objective
I have already completed work through approximately Phase 13 of my assessment playbook, including creating hooks. Do not assume every earlier step is complete or correct.
Your task is to:

1. Inspect the current project and identify exactly what is already implemented.
2. Preserve all correct existing work and avoid overwriting completed files.
3. Complete missing or incorrect Claude Code configuration and assessment features.
4. Finish the ASP.NET Core backend and React frontend as a functional application.
5. Connect the frontend to the backend and SQLite database.
6. Use the project's rules, skills, subagents, hooks, MCP integration, and Cognine plugin appropriately during actual implementation and verification.
7. Run the available automated tests and builds.
8. Produce honest assessment evidence, a working README, a demo script, and assessor Q&A.

Do not merely create configuration files that look correct. Demonstrate that the configured features actually work.
2. Mandatory safety rules

- Work only inside the assessment repository, except for explicitly permitted local Git worktree operations.
- Never access or modify the demo project beyond read-only inspection that has already been authorized.
- Do not discard existing changes, reset branches, force-push, or clean the Git working tree.
- Do not overwrite existing files without inspecting their contents first.
- Do not delete a database, migrations, user data, or Git history.
- Do not weaken, skip, or remove tests to obtain a passing result.
- Never invent command output, test counts, plugin availability, MCP connectivity, or evidence.
- Never put secrets or credentials in source control.

- Use explicit C# types instead of `var`, following the project rules.
- Follow the existing architecture, `CLAUDE.md`, path-scoped rules, skills, and security hooks.
- Keep API controllers thin; implement business rules in services.
- Use typed React/TypeScript code without `any`.
- Keep all HTTP requests in the frontend API layer.
- Do not introduce dependencies without explaining why and obtaining approval where the project rules require it.
- Before destructive, potentially irreversible, or broad changes, explain the impact and wait for my explicit approval.
- Preserve existing `.claude/settings.json`, enabled plugins, hook configuration, `.mcp.json`, Git changes, and working features unless a verified issue requires a change.
- Do not claim a hook or subagent worked unless its execution is visible in the actual results or logs.

3. Stage A — Read-only project audit
   Start in Plan Mode.
   Before changing anything, inspect:

- Current working directory and Git branch/status.
- `CLAUDE.md`.
- `docs/requirements.md`.
- `docs/architecture.md`.
- `docs/plugin-evaluation.md`.
- `docs/reuse-assessment.md`.
- `.claude/settings.json`.
- `.mcp.json`, if present.
- All files under `.claude/rules/`, `.claude/skills/`, `.claude/agents/`, and `.claude/hooks/`.
- Existing backend and frontend source files.
- Existing migrations, tests, `.http` samples, and assessment evidence.
- The current `README.md`, if present.

Use file listings and Git status to establish what actually exists. Do not infer completion from documentation alone.
Create a status matrix with these columns:
`Requirement | Expected artifact/behaviour | Actual evidence | Status | Remaining work`
Audit at least the following:

- Requirements and architecture documentation.
- Backend solution and API scaffolding.
- SQLite database, entities, EF Core configuration, migrations, and development seed data.
- `CLAUDE.md` and all six rules, including path-scoped frontmatter.
- Reusable `aspnet-endpoint-workflow` skill.
- Project-specific `expense-reporting` skill.

- `api-implementer` and `qa-reviewer` subagents.
- PreToolUse and PostToolUse hooks.
- Hook test results and logs.
- SQLite MCP server configuration and read-only restrictions.
- Cognine Marketplace and React plugin availability.
- Categories, expenses, budgets, summaries, and reports endpoints.
- React pages, typed API client, validation, and error handling.
- Backend and frontend automated tests.
- Git commits, verification reports, and demo evidence.

Distinguish `PASS`, `PARTIAL`, `MISSING`, `FAIL`, and `BLOCKED`.
For every PASS, identify concrete evidence. For example, a file proves existence, but a successful hook test proves execution.
Do not modify files during this stage.

Show more

Used 3 tools ▸

Stage A is done and I changed nothing. Two findings outweigh the rest. The project has no Git repository yet. And none of your Claude Code features has produced execution evidence so far: the pre-hook has never logged a run, and the Cognine plugin's hooks fail on this machine.

I can't build or run the backend from this Desktop chat. Neither my workspace nor the shell on your computer here has the .NET SDK, and NuGet is blocked from both. So I can't build or test the C# code, generate migrations, run your C# hooks, connect the MCP server, or run plugin commands. Your Windows machine has everything (.NET 10.0.100, dotnet-ef 10.0.11, sqlite3 3.53.4, Git 2.54, according to the P01 transcript). Stages C–H therefore have to run in Claude Code CLI there, which is also the only place the assessment evidence can come from.

Status matrix

#	Requirement	Expected	Actual evidence	Status	Rema
1	Git repository	`git init`, a commit per phase	There is no `.git` folder in the project	MISSING	`git i` comm chang
2	Requirements doc	FR/VR/BR/NFR IDs and a feature map	`docs/requirements.md` covers FR-01 to FR-20	PARTIAL	§8 stil name 5000
3	Architecture doc	Design plus resolved conflicts	`docs/architecture.md` §8 lists open conflicts C1, C3, C4 and C6	PARTIAL	Resol below the ol name
4	Reuse and plugin docs	Both exist	`reuse-assessment.md`; the transcript shows `plugin-evaluation.md` being written	PASS (docs)	The re name don't
5	Backend scaffold	Solution, API and test projects; port 5200	`.sln`, net10 projects, EF 10.0.11, `http` profile on 5200	PARTIAL	`Progr` templ redire Proble No `Treat` (NFR- verifie
6	Data layer	Entities, DbContext, Fluent config, migration, seed data	No `Domain/`, `Data/` or `Migrations/` folder, and no `.db` file	MISSING	All of
7	CLAUDE.md	Accurate, verified statuses	Sections 1–9 exist; rules marked active, everything else "planned"	PARTIAL	Contr (D1). has nc
8	Six rules	5 path-scoped, `security.md` global	All 6 files with correct `---` / `paths:` frontmatter. `backend/**/*.cs`, `frontend/src/**` and the test-project paths match real files; Controllers/Contracts/Data/Domain don't exist yet	PARTIAL	The P no evi

#	Requirement	Expected	Actual evidence	Status	Rema
9	`aspnet-endpoint-workflow` skill	SKILL.md and checklist.md	No `.claude/skills/` folder	MISSING	Creat
10	`expense-reporting` skill	SKILL.md	No `.claude/skills/` folder	MISSING	Creat
11	`api-implementer` agent	Edit-capable, backend only	No `.claude/agents/` folder	MISSING	Creat
12	`qa-reviewer` agent	No Edit/Write tools	No `.claude/agents/` folder	MISSING	Creat
13	Pre-hook	File, registration, tested	`pre-guard.cs` (22.5 KB) is registered for `PreToolUse` with matcher `Bash|Edit|Write` and timeout 30. The code is sound: explicit types, fail-open, protects the demo folder, logs every decision	PARTIAL	Never `.clau` and it it has handl but th includ `query`
14	Post-hook	`post-verify.cs` plus `PostToolUse` registration	Neither exists	MISSING	Creat
15	Hook tests and logs	8 payload tests and a live block	No `tests/` folder and no log	MISSING	P14 ar
16	SQLite MCP	`.mcp.json`, snapshot, write tools denied	No `.mcp.json`	MISSING	See th
17	Cognine plugin	Installed and used	`enabledPlugins` is set in `settings.json`; the transcript shows "✓ Installed React Coding Standards" with 9 skills and 3 commands	PARTIAL	No au **plugi** transc "Post" error . found `pytho`

#	Requirement	Expected	Actual evidence	Status	Rema
18	Categories, expenses, budgets, summaries, reports	Endpoints FR-01 to FR-16	No controllers or services	MISSING	All of
19	ProblemDetails, CORS, OpenAPI	Errors, CORS for 5173, docs plus UI	Only the template's `AddOpenApi()`	PARTIAL	Every
20	`.http` samples	Every endpoint	Only the host variable line	PARTIAL	All th
21	Backend tests	Unit and integration	Test project and packages exist, with 0 test files	MISSING	All of
22	Frontend scaffold	Vite react-ts, Vitest, `/api` proxy	Everything set up, plus a smoke test; `dist/` shows one successful build	PASS (scaffold)	The te evide
23	Frontend features	types, api, hooks, pages	Only `App.tsx` with a heading	MISSING	All of
24	Evidence, README, demo	verification, evidence, audits, demo script	`docs/evidence/` is empty and none of the files exist	MISSING	Stage
25	Repo hygiene	Clean root	`AssessmentChat.tsx` (51 KB) at the root is a session transcript, not code	Note	Renar `docs/` `01-tr` (need

I inspected the published `mcp-sqlite@1.0.9` package directly. Its tools are `db_info`, `query`, `list_tables`, `get_table_schema`, `read_records`, `create_record`, `update_records` and `delete_records`. The `query` tool executes any SQL, writes included. Least privilege therefore needs three layers: deny the three write tools, add a pre-guard rule that lets `query` run only SELECT/WITH/PRAGMA, and point the server at a snapshot copy of the database.

Contradictions that need your decision

#	Conflict	My recommendation
D1	DB file: your prompt says `expense-management.db`, but CLAUDE.md, requirements and architecture say `expenses.db`	Use `expense-management.db` as your prompt says, and update the three docs
D2	Startup migration: playbook P22 says `Database.Migrate()` at startup; CLAUDE.md, the data rule and FR-17 say no migration at startup, use `dotnet ef database update`	**Keep the repo rule.** It's stricter and already written into a rule file
D3	Summary and report endpoints: your prompt says `/summaries/monthly?year&month` and `/reports/expenses?from&to&categoryId&format`; FR-13 to FR-16 say `/summaries/monthly?year`, `/summaries/categories?month` and `/reports/monthly?month&format`. The CSV header differs too (with or without Notes)	Follow `requirements.md`. It's the written contract and has acceptance criteria
D4	Budget API body: FR-10 says `{categoryId, month:"YYYY-MM", limit, type}`; the playbook says year, month, limitAmount, enforcement	**Follow FR-10 for the API.** The entity keeps Year, Month, LimitAmount and Enforcement, as architecture C1 already says
D5	Requirements base URL is 5000; everything else uses 5200	Fix the doc to 5200
D6	The OpenAPI UI needs a new NuGet package	Approve `Scalar.AspNetCore`, or skip it and rely on `/openapi/v1.json` plus the `.http` file
D7	Feature names: requirements §8 says `rest-endpoint-tests`, `code-reviewer`, `api.md` etc.; CLAUDE.md and the playbook say `aspnet-endpoint-workflow`, `api-implementer`, the 6 rule files	**Use the CLAUDE.md names** and update requirements §8 and the architecture tree
D8	Post-hook scope: requirements §8 says `dotnet format` plus build; the playbook says build only	**Build only** (`dotnet format` adds about 10 s per edit), and update §8
D9	The plugin hooks need a real `python3`	Your action, outside the repo: Windows Settings → Apps → Advanced app settings → App execution aliases → turn off `python3.exe`, then check `python3 --version`

Remediation plan

Order	Stage	Work	Verified by
0	Safety	`git init`, baseline commit; move the transcript file	`git log`
1	C: Hooks	Add the `PowerShell` matcher and MCP `query` guard to pre-guard; create post-verify; run P14 and P16	8/8 payload tests and `hooks.log`
2	C: Skills and agents	Create the 2 skills and 2 agents	`/skills`, `/agents`, a qa-reviewer trial run
3	D: Data	Entities, configurations and seed data via api-implementer; migration with `dotnet ef`; `database update`	sqlite3 `.tables`
4	C: MCP	Snapshot, `.mcp.json` pinned to 1.0.9, deny rules, reviewer tools	`/mcp` connected, SELECT works, write denied
5	D: API	Foundation → categories → expenses → list → budgets → summaries → reports, each followed by a qa-reviewer pass	`dotnet test` per feature
6	E: Frontend	types/api → pages → tests; then plugin audits before and after	`npm test`, `npm run build`, audit files
7	F: Integration	OpenAPI contract check, live UI, persistence, MCP total cross-check	`docs/verification.md`
8	G/H: Evidence	claude-code-evidence, README, demo script, Q&A, final review of diff and status	All rows backed by evidence

The risks are a slow first run of each C# hook (it compiles), `TreatWarningsAsErrors` producing many errors in the first build, and usage limits if you run two sessions.

How to proceed

- **Option A (recommended):** paste this same prompt into Claude Code CLI in `claude-code-workshop-assessment` and approve the plan there, along with your answers to D1–D9. Everything it runs then becomes real evidence.
- **Option B:** I write the configuration and code files from here, using exactly the answers you give to D1–D9. You then run the builds, tests and demos in Claude Code CLI, and it fixes what fails. That's faster, but the C# would be unverified until your machine compiles it, and the evidence would still have to come from the CLI.

Reply with A or B, plus "approve as recommended" or your changes to D1–D9.

AICC102 Expense Management API — Stage B Onward: Implementation and Verification

You are the lead software engineer responsible for completing my AICC102 Claude Code assessment project using the files and tools available in this Claude Desktop session.

1. Working directory and permissions

My assessment project is located at:

`C:\Users\Laxmi Bandaru\claude-code-workshop-assessment`

The separate demo project is located at:

`C:\Users\Laxmi Bandaru\claude-code-workshop`

The demo project is strictly read-only. Never create, edit, move, delete, build, install, or run Git commands inside it.

First, verify that you can access the assessment folder and identify the actual current path. Use only tools available in this Claude Desktop session. Do not claim access to a terminal, filesystem, or tool that is not actually available.

I have completed Stage A, the read-only audit. Use the audit status matrix and findings I provide or that are available in the project documentation as the starting point. Do not repeat the entire audit unnecessarily. Verify specific files again when needed before modifying them.

2. Main objective

Complete the remaining assessment in stages:

1. Stage B — finalise the remediation plan and resolve implementation decisions.
2. Stage C — complete and verify Claude Code configuration, rules, skills, subagents, hooks, MCP, and plugin integration.
3. Stage D — complete the ASP.NET Core backend, SQLite persistence, and backend tests.
4. Stage E — complete the React frontend and frontend tests.
5. Stage F — integrate and verify the frontend and backend.
6. Stage G — create accurate documentation and assessment evidence.
7. Stage H — perform final verification and Git hygiene.

Do not rebuild files or features that are already correct. Implement missing or defective functionality only.

3. Important limitation: Claude Desktop versus Claude Code CLI

This assessment requires proof that Claude Code-specific features actually executed.

Claude Desktop and Claude Code CLI are not automatically the same environment. Do not claim that Claude Code hooks, slash commands, subagents, plugin commands, MCP permissions, or configuration loading have been tested merely because files were created or edited in Desktop.

Before implementation, identify which operations can actually be executed from this session.

- Use Desktop file access for inspecting and editing project files where available.
- Use an available terminal or execution tool for commands only if the tool genuinely exists and has access to the assessment folder.
- Use Claude Code CLI for verification steps that require its actual runtime, such as loading skills, running subagents, executing hooks, checking plugin commands, and verifying Claude Code MCP permissions.
- Record Desktop-authored changes separately from features demonstrated in Claude Code CLI.
- Never fabricate command output, test results, logs, or evidence.

If a required operation cannot be performed from Desktop, continue with work that can safely be completed, and maintain a clear list of CLI verification tasks. Do not describe the whole project as verified until those checks pass.
4. Safety and implementation rules

- Preserve the existing branch, working changes, configuration, and completed work.
- Do not discard changes, reset branches, force-push, or rewrite Git history.
- Inspect existing files before overwriting them.
- Never delete an existing database or migration history.
- Do not weaken tests or remove requirements to make tests pass.
- Do not expose secrets or credentials.
- Use explicit C# types instead of `var`.
- Keep API controllers thin and business logic in services.
- Use strict, typed React/TypeScript without `any`.
- Keep frontend HTTP requests in the API layer.
- Follow the existing `CLAUDE.md`, rules, skills, architecture, and requirements.
- Do not introduce new dependencies without explaining their purpose and obtaining approval when required.
- Do not make destructive or irreversible changes without explicit approval.
- Do not commit unrelated user changes.
- Do not silently change the agreed API contract.

If a task is blocked, explain the exact blocker, its impact, and the smallest safe next step.
5. Stage B — finalise the remediation plan
Use the completed Stage A findings to prepare a prioritised plan.
For each stage, identify:

- Objective and dependencies.
- Files to create or modify.
- Existing files that must be preserved.
- Implementation tasks.
- Risks and required decisions.
- Automated validation commands.
- Required Claude Code CLI verification.
- Acceptance criteria.

- Expected Git milestone.

Use the existing `docs/requirements.md` as the API contract and the architecture document for design decisions.
Apply these previously recommended decisions unless the existing project evidence reveals a material conflict:

- Database filename: `expense-management.db`.
- Keep the existing stricter rule against automatic migration at application startup. Apply migrations explicitly through `dotnet ef`.
- Follow the endpoint and response contract in `docs/requirements.md`.
- Budget API requests use the established requirements contract; internal domain properties may retain `Year`, `Month`, `LimitAmount`, and `Enforcement`.
- API base URL: `http://localhost:5200`.
- Use the existing Claude Code feature names: `aspnet-endpoint-workflow`, `expense-reporting`, `api-implementer`, and `qa-reviewer`.
- Post-hook scope: build-check changed C# files, as agreed in the project plan.
- Preserve plugin settings. Diagnose the Cognine plugin's Python hook issue rather than bypassing security checks.
- Obtain explicit approval before adding `Scalar.AspNetCore` or another new package if approval is required by the existing rules.

Show me the finalised plan and any decisions that cannot be safely resolved from existing documentation. Wait for approval before broad implementation.

Show more

Used 2 tools ▸

Stage B plan below. Nothing has been changed. I checked access again first, and the project has moved on since the audit:

- **Git is set up.** You're on `main` with one commit, `d222e5c chore: baseline AICC102 scaffold, docs and Claude Code config`. The only untracked file is `AssessmentChat.tsx`.
- **The pre-hook runs in Claude Code.** `.claude/logs/hooks.log` has 7 real `pre-guard | ALLOW | Bash` entries (10:09–10:12 UTC). The "allow" path is proven; the "block" path isn't yet.

What this Desktop session can and can't do

Can do	Can't do
Read and edit files in `claude-code-workshop-assessment` through the shell on your computer	Run `dotnet` (not installed in either of my environments, and NuGet is blocked), so no backend build, test, migration or C# hook run
Build and test the frontend in my cloud workspace (npm is reachable there)	Use your `frontend/node_modules`: it holds Windows-only binaries, so I test a separate copy
Read `hooks.log` and `git log` as evidence	Run `git commit`: from here it leaves `.lock` files behind in `.git`. You commit in the CLI
Inspect the `mcp-sqlite` package	Load skills or agents, fire hooks, check `/mcp`, apply permissions or run plugin commands. Only the Claude Code CLI can do those

So I write the files ("D" below), and you verify them in Claude Code CLI ("CLI"). Everything I author stays "authored, not verified" in the evidence docs until a CLI step passes.

Plan by stage

C1. Hooks

- Depends on: nothing.
- Files:
  - Modify `.claude/hooks/pre-guard.cs`: add an MCP rule that lets the `expense-db` `query` tool run only SELECT/WITH/PRAGMA, and blocks `create_record`, `update_records` and `delete_records`.
  - Modify `.claude/settings.json`: matcher becomes `Bash|PowerShell|Edit|Write|mcp__expense-db__.*`; add `PostToolUse`.
  - Create `.claude/hooks/post-verify.cs` and the 8 payload files in `.claude/hooks/tests/*.json`.
  - Preserve: the existing guard logic, `enabledPlugins`, `hooks.log`.
- Risks: first-run compile time on each hook; your path contains a space.
- CLI checks: new session, `/hooks`, the P14 payload table (8/8), the live DROP TABLE prompt being blocked, and the P16 HookProbe test (post-verify reports FAILED, then OK).
- Done when: the log shows BLOCK, ALLOW, FAILED and OK entries.
- Commit: `Phase 5: pre and post hooks`.

C2. Skills and agents

- Depends on: C1.
- Files to create:

  - `.claude/skills/aspnet-endpoint-workflow/SKILL.md` and `checklist.md`.
  - `.claude/skills/expense-reporting/SKILL.md`. It follows the FR-13 to FR-16 contract, and its CSV header is `Date,Title,Category,Amount` as in FR-16.
  - `.claude/agents/api-implementer.md` and `.claude/agents/qa-reviewer.md` (tools: Read, Grep, Glob, Bash, no Edit or Write).
- CLI checks: `/skills`, `/agents`, the P19 question, and the P21 qa-reviewer trial with its "can you edit?" proof.
- Commit: `Phase 6: skills and subagents`.

D1. Data layer

- Depends on: C2.
- Files (D):
  - `Domain/*`: Category, Expense, Budget, BudgetEnforcement, BudgetStatus.
  - `Data/AppDbContext.cs`, `Data/Configurations/*` and `Infrastructure/DevDataSeeder.cs`.
  - `appsettings.json` with `Data Source=expense-management.db`.
  - `Program.cs` rewritten (it's still the template).
  - `TreatWarningsAsErrors` added to the API project file.
- CLI steps: delegate review and fixes to api-implementer; run `dotnet build backend`; create the migration with `dotnet ef migrations add InitialCreate`; apply it with `dotnet ef database update`; check with `sqlite3 … .tables`.
- Risk: the first build under `TreatWarningsAsErrors` may need fixes. The post-verify hook feeds those errors back to Claude to correct.
- Commit: `Phase 7: data layer and migration`.

C3. MCP

- Depends on: D1 (the database must exist).
- Files (D):
  - `.mcp.json`: `cmd /c npx -y mcp-sqlite@1.0.9 .claude/mcp/expense-snapshot.db`.
  - `permissions.deny` for the 3 write tools.
  - The qa-reviewer gets the read tools: `query`, `list_tables`, `get_table_schema`, `db_info`, `read_records`.
  - The snapshot refresh command goes into CLAUDE.md.
- CLI checks: create the snapshot, restart, `/mcp` shows connected, then P26: schema and SELECT queries work and an INSERT is blocked.
- Risk: the package's native `sqlite3` module must install on Windows.
- Commit: `Phase 8: MCP`.

D2. API

- Depends on: D1 and C3.

- Files (D):
  - Foundation: `ServiceResult`, `ServiceResultMapper`, ProblemDetails and exception handler, CORS, `TimeProvider`.
  - Validators, `BudgetCalculator` and `CsvWriter`.
  - Five services and five controllers (categories, expenses, budgets, summaries, reports).
  - Every route in the `.http` file.
- Contract: `docs/requirements.md` FR-01 to FR-18 exactly. Budget requests use `{categoryId, month:"YYYY-MM", limit, type}`. Reports are `/api/reports/monthly?month&format`.
- CLI: build and test after each feature, P29 qa-reviewer loop, P33 MCP total cross-check.
- Done when: every FR-01 to FR-16 acceptance criterion has a passing test.
- Commits: one per feature.

D3. Backend tests

- Built alongside D2.
- Files (D): `Integration/TestApiFactory.cs` (in-memory SQLite, `Testing` environment, a fixed `TimeProvider`), plus `Unit/*` and `Integration/*`, all tagged with `[Trait("Requirement", …)]`.
- CLI: `dotnet test backend` with the real counts.

E. Frontend

- Depends on: the D2 contract. It can start in parallel because the contract is written down.
- Files (D): `src/types`, `api`, `hooks`, `components` and `pages`, plus tests. `App.tsx` and `App.test.tsx` are modified; the Vite and tsconfig settings are preserved.
- D-verified: I run `npm test` and `npm run build` on my cloud copy.
- CLI: rerun them on Windows, then P41 plugin audits before and after.
- Blocker: the plugin hooks need a real `python3` (decision 5).
- Commits: `Phase 10: frontend`, `Phase 11: plugin audit fixes`.

F. Integration

- CLI only: API and Vite running together, compare against `/openapi/v1.json`, click through the UI, restart to check persistence, cross-check MCP totals.
- Output: `docs/verification.md`.

G. Evidence and docs

- Files (D, filled from CLI results): `docs/claude-code-evidence.md`, `docs/demo-script.md`, `README.md`, and `docs/evidence/plugin-audit-*.md`. Update CLAUDE.md (DB name, statuses), requirements (base URL 5200, §8 feature names, post-hook scope) and the architecture conflict table.

- "Planned" labels are removed only after a CLI check passes.

H. Final checks

- CLI: `git status` and `git diff` review; confirm no `*.db`, `bin`/`obj`, `dist` or `.claude/logs` files are tracked; final build and test run; final commit.

Decisions I still need from you

#	Decision	My recommendation
1	OpenAPI UI: add `Scalar.AspNetCore`?	**Yes:** it's one package and Development-only. I can't check its version from here, so I'd pin a current 2.x release and you confirm with `dotnet add package Scalar.AspNetCore`. If you say no, docs are `/openapi/v1.json` plus the `.http` file only
2	Seed data: FR-17 asks for seeded categories, budgets and expenses, but tests apply the same migrations. `HasData` inside the migration would also seed the test databases	**Seed nothing in the migration.** Add a `DevDataSeeder` that runs at startup only in Development when the database is empty. FR-17's "API starts → categories returned" still holds, and tests stay clean. I'd update the FR-17 wording
3	`AssessmentChat.tsx` at the repo root, a session transcript with a `.tsx` extension	Move it to `docs/evidence/session-01-transcript.txt` (a move, not a delete)
4	`TreatWarningsAsErrors` (NFR-06) on the API project	**Keep it** as the requirements say. The first build may need a few fixes
5	Cognine plugin hooks fail because `python3` is the Windows Store stub	Your action, outside the repo: Settings → Apps → Advanced app settings → App execution aliases → turn off `python3.exe` and `python.exe`, then check `python3 --version` works in Git Bash. If `python3` still isn't found, it's a PATH issue: tell me and we'll look at it. I won't bypass or edit the plugin
6	Workflow	I author the files, then send you the exact CLI prompts for each stage. You run them and paste the results, which go into the evidence

Reply "approve" to accept all six recommendations, or tell me which to change. I'll then start C1 (the hook changes and `post-verify.cs`) and send you its CLI verification prompts.

6. Stage C — Claude Code configuration and assessment features

Inspect and complete only what is missing.

CLAUDE.md and rules
Verify that `CLAUDE.md` accurately describes the application, commands, ports, database, architecture, business rules, available tools, and definition of done.
Verify these six rules:

- `.claude/rules/dotnet-backend.md`
- `.claude/rules/api-design.md`
- `.claude/rules/data-and-migrations.md`
- `.claude/rules/react-typescript.md`
- `.claude/rules/testing.md`
- `.claude/rules/security.md`

The first five must have appropriate path-scoped frontmatter. The security rule must apply globally.
Check their patterns against actual project files and verify loading using Claude Code CLI. A file existing on disk is not proof that Claude Code loaded it.
Skills
Verify and complete:

- `.claude/skills/aspnet-endpoint-workflow/SKILL.md`
- `.claude/skills/aspnet-endpoint-workflow/checklist.md`
- `.claude/skills/expense-reporting/SKILL.md`

Use the reusable ASP.NET skill during endpoint implementation. Use the expense-reporting skill for month boundaries, decimal calculations, summaries, CSV formatting, budget rules, and database cross-checks.
Subagents
Verify and complete:

- `.claude/agents/api-implementer.md`
- `.claude/agents/qa-reviewer.md`

The implementation agent must be restricted to backend implementation responsibilities. The reviewer must not have Edit or Write tools and must review code independently.
Use the implementation agent for appropriate backend tasks and invoke the reviewer after each meaningful backend feature. Record actual results.
Hooks
Inspect the existing `.claude/hooks/pre-guard.cs` and hook registration before changing anything.
Complete:

- `.claude/hooks/pre-guard.cs`
- `.claude/hooks/post-verify.cs`
- `.claude/settings.json` registrations.
- Controlled hook tests.
- `.claude/logs/hooks.log`.

Test malformed inputs, allowed operations, blocked operations, exit codes, logging, and relevant file protections using safe test payloads. Do not perform real destructive operations to demonstrate blocking.
Ensure the pre-hook protects the demo project and applies the agreed safety restrictions. Ensure the post-hook verifies changed C# and TypeScript files according to the approved plan.
SQLite MCP
Inspect the actual MCP server version and exposed tool names.
Configure `expense-db` to use a snapshot at `.claude/mcp/expense-snapshot.db`, where feasible. Restrict access to read-only database inspection.
Deny all write-capable tools using the permission syntax supported by the installed Claude Code version.
Verify connectivity, schema inspection, and SELECT queries through Claude Code CLI. Verify that write attempts are blocked by actual permissions.
Document safe snapshot refresh steps.
Do not claim that a server is connected until a real connection test succeeds.
Cognine plugin
Verify the actual Marketplace configuration, plugin installation, available commands, and hook behaviour.
Run available React architecture, security, and accessibility audits. Save genuine before-and-after findings. Fix genuine high-priority findings and rerun tests.
If the Python hook fails because `python3` resolves to a Windows Store alias, report the precise failure and provide the required Windows-side remediation.
Do not disable security checks merely to make the plugin appear functional.

Show more

approve

Approved; starting Stage C. First I'll check every place the old database name and port appear, before editing anything.

Used 3 tools ▸

Your Claude Code CLI session is changing files at the same moment (`pre-guard.cs` was modified and hook tests were added). I'll look at what it changed before touching those files.

Ran a command on your computer ▸

I've stopped before touching any hook or settings file. **Your Claude Code CLI session is running the hook tests right now, in the same folder,** so if both of us keep editing we'll overwrite each other's work.

What the CLI session is doing (from the files and the log):

- It added a `PRE_GUARD_LOG_FILE` option to `pre-guard.cs` (+21/−5 lines), so test runs log to a temp file instead of the real `hooks.log`. The change looks correct.
- It created `.claude/hooks/tests/test-pre-guard.sh` and `docs/evidence/hooks/pre-guard-tests.txt`. So far that file shows 21 PASS cases, including the blocks for `rm -rf`, `git reset --hard` and `git push --force`.
- `hooks.log` now also records an invalid-JSON fail-open, plus Edit and Write calls going through the hook. That's real execution evidence.

What I changed (small, already on disk, nothing committed):

- Replaced `expenses.db` with `expense-management.db` in `CLAUDE.md`, `security.md`, `testing.md`, `docs/architecture.md` and `docs/requirements.md`, and set the requirements base URL to 5200 (decisions D1 and D5).
- Moved `AssessmentChat.tsx` to `docs/evidence/session-01-transcript.txt` (decision 3).

Your CLI session's `git status` will show these. They don't touch the files it's editing.

Which session should own the rest of Stage C?

1. **CLI owns hooks and MCP; Desktop writes the skills and agents (recommended).** The CLI finishes P14, then P15–P16 (`post-verify.cs`), and keeps sole ownership of `.claude/hooks/` and `.claude/settings.json`. Meanwhile I write the two skills and two agents, which the CLI isn't touching, update the CLAUDE.md workflow table, and tell you when they're on disk. You then run P19 and P21 in the CLI to verify them.
2. **CLI owns all of Stage C.** I stop editing and only give prompts and review results.
3. **Desktop owns Stage C.** You pause the CLI session after the current test run, and I write `post-verify.cs` and the MCP/settings changes.

Reply 1, 2 or 3. Until then I won't edit anything.

AICC102 — Stage C Audit, Completion and Verification
You are responsible for verifying and completing Stage C of my AICC102 Expense Management API assessment.
Project directory: `C:\Users\Laxmi Bandaru\claude-code-assessment`
Demo project: `C:\Users\Laxmi Bandaru\claude-code-workshop` — STRICTLY READ-ONLY.
Objective
Determine whether Stage C is fully completed. Inspect the actual project files and available execution evidence. Then implement only missing or defective items, preserve working configuration, and verify each feature using real results. Do not assume Stage C is complete merely because files exist. Distinguish configuration, successful loading, actual execution, and verified behaviour.

Phase 1 — Audit before editing
Inspect the following:

1. `CLAUDE.md`
2. `.claude/settings.json`
3. All six rules under `.claude/rules/`
4. `.claude/skills/aspnet-endpoint-workflow/SKILL.md`
5. `.claude/skills/aspnet-endpoint-workflow/checklist.md`
6. `.claude/skills/expense-reporting/SKILL.md`
7. `.claude/agents/api-implementer.md`
8. `.claude/agents/qa-reviewer.md`
9. `.claude/hooks/pre-guard.cs`
10. `.claude/hooks/post-verify.cs`
11. Existing hook tests and `.claude/logs/hooks.log`
12. `.mcp.json`, MCP configuration, permissions, and snapshot files
13. Cognine plugin configuration, installation evidence, commands, and hook errors
14. Relevant Git status and recent commits
15. Existing Stage C documentation and evidence

Verify the actual project directory before inspecting files. Do not modify the demo project.
Create a status matrix:
`Feature | Required behaviour | Existing files/configuration | Execution evidence | Status | Required action`
Use these statuses:

- `PASS`: implemented and supported by real verification evidence.
- `PARTIAL`: some requirements work, but others are missing or unverified.
- `MISSING`: required implementation is absent.
- `FAIL`: an attempted verification demonstrated a defect.
- `BLOCKED`: verification cannot proceed because a dependency, permission, or required environment is unavailable.

For every finding, provide the file path, relevant configuration or code evidence, and the verification needed.
Do not edit anything during this phase.
Phase 2 — Verify the required behaviour
A. CLAUDE.md and rules
Verify these files:

- `.claude/rules/dotnet-backend.md`
- `.claude/rules/api-design.md`
- `.claude/rules/data-and-migrations.md`
- `.claude/rules/react-typescript.md`
- `.claude/rules/testing.md`
- `.claude/rules/security.md`

Check the YAML frontmatter and confirm the first five rules have appropriate `paths:` patterns matching the actual project files. Confirm `security.md` applies globally.
Check that `CLAUDE.md` documents the correct ports, database filename, run/test commands, architecture, business rules, and actual availability of the other Claude Code features.
Use the installed Claude Code CLI to verify rule loading when available. Do not treat a file inspection or a statement in `CLAUDE.md` as proof that a rule loaded.
B. Skills
Verify:

- The ASP.NET endpoint skill is reusable and not tied to the expense domain.
- Its checklist contains a practical endpoint implementation and verification workflow.
- The expense-reporting skill documents the application's budget rules, month boundaries, decimal calculations, SQLite limitations, summaries, CSV rules, and MCP cross-checks.
- The skills are actually invoked or otherwise demonstrably used through the supported Claude Code workflow.

Record actual invocation evidence. If CLI access is unavailable, mark runtime verification as `BLOCKED` or `NOT DEMONSTRATED`.
C. Subagents
Verify:

- `api-implementer` is restricted to backend implementation.
- `qa-reviewer` is an independent reviewer and has no Edit or Write tools.
- The agent definitions use valid configuration for the installed Claude Code version.

Where CLI access is available, invoke the appropriate agent on a small, non-destructive task. Capture its actual response and verify that the reviewer did not modify files.
Do not claim an agent was used if only its configuration file was inspected.
D. Hooks
Inspect the existing pre-hook and settings before changing them.
Verify:

- PreToolUse and PostToolUse registration and matchers.
- Correct hook commands, quoting, timeouts, paths, and exit-code behaviour.
- Protection of the demo project and other required safety restrictions.
- Logging to `.claude/logs/hooks.log`.
- Handling of malformed input and log failures.
- Post-hook checks for changed C# and TypeScript files.

Run safe, controlled hook tests for allowed operations, blocked operations, malformed payloads, exit codes, and logging. Use test inputs and temporary fixtures that cannot damage real data. Never execute a genuinely destructive command to demonstrate that it would be blocked.
If the pre-hook has no execution evidence, run it through the actual Claude

Code hook mechanism where available. Do not count manually running the C# file as proof that Claude Code invoked it.
If a matcher omits a relevant tool or operation, explain the impact and propose a minimal correction. Preserve the existing plugin settings.
E. SQLite MCP
Inspect the actual `.mcp.json`, package/version configuration, available tools, and current permissions.
Verify that:

- The server is named `expense-db`.
- The server uses a snapshot at `.claude/mcp/expense-snapshot.db`, if the required database can safely be created.
- Database inspection is read-only.
- Every write-capable tool is denied using permission syntax supported by the installed Claude Code version.
- Any unrestricted SQL query tool is restricted appropriately, or the configuration prevents unsafe access through a verified alternative.
- The `qa-reviewer` receives only necessary read-only tools if the installed version supports that restriction.

Use the real CLI `/mcp` command or equivalent supported diagnostic to verify connectivity. Run a schema inspection and SELECT query through the connected server. Verify that a write attempt is denied by configured permissions without modifying the real database.
Do not claim connectivity from the presence of `.mcp.json` alone. If the database snapshot does not yet exist, explain whether the MCP setup can be completed now or must wait for the database stage.
F. Cognine Marketplace plugin
Verify the actual installed/enabled state of `react-coding-standards-best-practices`.
Inspect the actual commands and hook registrations rather than assuming their names. Investigate any reported Python errors, including whether `python3` resolves to a Windows Store execution alias.
Where the plugin works, run the available architecture, security, and accessibility audits against the actual frontend. Save genuine before-and-after evidence.
Do not disable security hooks simply to make the plugin pass. If Windows configuration changes are needed, provide the exact recommended steps and wait for me to perform them when you cannot safely change them yourself.
Phase 3 — Report findings and propose corrections
After completing the read-only audit and any safe verification that requires no modifications, show me:

1. The completed status matrix.
2. A list of missing or defective files.
3. A list of features configured but not demonstrated.
4. Exact blockers requiring Claude Code CLI or Windows action.
5. A prioritised correction plan.

6. The files each correction would affect.
7. The tests or commands that will prove the correction works.

Do not start broad edits until I approve the correction plan.
Phase 4 — Implement approved corrections
After approval:

- Modify only missing or defective Stage C files.
- Preserve valid settings and existing plugin configuration.
- Do not recreate already-correct skills, rules, or agents.
- Create or repair hook tests and logging as needed.
- Configure MCP least privilege using actual supported tool names and permissions.
- Use actual Claude Code CLI execution for features requiring CLI verification.
- Capture genuine command output and results.
- Re-run the relevant checks after every correction.
- Update `CLAUDE.md` and evidence documentation to reflect verified reality.

If a feature cannot be verified in the current environment, label it `BLOCKED` or `NOT DEMONSTRATED`. Do not fabricate a successful result.
Phase 5 — Final Stage C report
At the end, provide a table:
`Feature | Final status | Files changed | Verification performed | Actual result | Remaining work`
Stage C is complete only when every required feature has the necessary implementation and verification evidence. Configuration-only features must not be described as fully verified.
Start now with Phase 1. Audit only. Do not edit files until the audit and correction plan have been presented and approved.

Show more

Used 2 tools ▸

Phase 1 is done and nothing was edited. One correction first: the project folder is `C:\Users\Laxmi Bandaru\claude-code-workshop-assessment`, not `…\claude-code-assessment` (that path doesn't exist). Your CLI session has meanwhile produced real hook evidence, so the pre-hook is now the strongest part of Stage C.

Status matrix

Feature	Required behaviour	Existing files / config	Execution evidence	Stat
Project and Git	Correct folder; committed work	`main` at `d222e5c` (baseline). Uncommitted: the CLI's `pre-guard.cs` change, `.claude/hooks/tests/`, `docs/evidence/`, and my doc fixes	`git status`	PAR
CLAUDE.md	Accurate ports, DB, commands, architecture, rules, feature status	Ports 5200/5173 ✓, `expense-management.db` ✓ (fixed earlier), no startup migration ✓, business rules ✓	No `/memory` or P10 evidence	PAR
Rules (6)	5 path-scoped, `security.md` global	All 6 files with valid `---` / `paths:` frontmatter; `security.md` has no `paths:` ✓. `backend/**/*.cs` matches `Program.cs`, `frontend/src/**` matches `App.tsx`, the test-project path matches. `Controllers/`, `Contracts/`, `Data/`, `Domain/` and `Migrations/` don't exist yet	No loading evidence	PAR
`aspnet-endpoint-workflow` skill + checklist	Reusable, practical workflow	No `.claude/skills/`	—	MIS
`expense-reporting` skill	Project rules, CSV, MCP cross-check	—	—	MIS

Feature	Required behaviour	Existing files / config	Execution evidence	Stat
`api-implementer` agent	Backend-only editor	No `.claude/agents/`	—	MIS
`qa-reviewer` agent	No Edit or Write tools	—	—	MIS
Pre-hook	Blocks destructive commands and protected files; fails open; logs	`pre-guard.cs` (22.9 KB), registered as `PreToolUse` with matcher `Bash|Edit|Write` and timeout 30. The CLI added a `PRE_GUARD_LOG_FILE` override for tests; log write errors are caught	**Real Claude Code runs in** `hooks.log`: `10:16:55 BLOCK Bash rm -rf…`, `10:17:00 BLOCK Write '.env.hookprobe'…`, plus ALLOW lines for Bash, Edit and Write. **Controlled tests:** `docs/evidence/hooks/pre-guard-tests.txt`, **59 passed, 0 failed** (demo folder, `.db` files, Migrations, `.env`, fail-open, log count)	PAS (gap belo
Post-hook	Build-check `.cs`, type-check `.ts(x)`	No `post-verify.cs`; no `PostToolUse` entry in `settings.json`	—	MIS
Hook test suite	Repeatable tests	`.claude/hooks/tests/test-pre-guard.sh` (uncommitted)	59/59	PAS hool

Feature	Required behaviour	Existing files / config	Execution evidence	Stat
SQLite MCP	`expense-db`, snapshot, read-only, write tools denied	No `.mcp.json`, no `.claude/mcp/`, no database	—	BLC unti Stag migr crea DB
Cognine plugin	Installed, commands run, hooks work, audits saved	`enabledPlugins` set; the transcript shows "✓ Installed", 9 skills, 3 commands	The transcript shows 3× "PostToolUse:Write hook error … Python was not found"	FAIl (hoc audi MIS
Stage C evidence docs	Recorded results	`docs/evidence/hooks/pre-guard-tests.txt`, `docs/evidence/session-01-transcript.txt`	—	PAR

Missing files: `.claude/skills/aspnet-endpoint-workflow/SKILL.md` and `checklist.md`, `.claude/skills/expense-reporting/SKILL.md`, `.claude/agents/api-implementer.md`, `.claude/agents/qa-reviewer.md`, `.claude/hooks/post-verify.cs`, a post-hook test script, and `.mcp.json`.

Configured but not demonstrated: CLAUDE.md loading (P10), rule path-scoping (P12), and the Cognine plugin's audit commands.

Blockers

Blocker	Needs	Smallest next step
Plugin hooks need a real `python3`	You, on Windows	Settings → Apps → Advanced app settings → App execution aliases → turn off `python.exe` and `python3.exe`. In a new Git Bash run `python3 --version`. If it isn't found, the real Python isn't on PATH: run `py -0p` and tell me the path
MCP needs a database	Stage D1 (migration)	Configure MCP right after `dotnet ef database update`
Runtime checks (hooks, `/skills`, `/agents`, `/mcp`, rule loading, plugin commands)	Claude Code CLI	Prompts below. I can't run them from Desktop
Building `post-verify.cs`	.NET (CLI only)	The first live run compiles it

Correction plan, in order

#	Correction	Files	Who	Proof
1	Commit the current work	—	CLI	`git log` sh a "Phase 5a: pre-guard te commit
2	Add `PowerShell` to the pre-hook matcher	`.claude/settings.json` (one line; `enabledPlugins` unchanged)	Desktop	New session `/hooks` sho the matcher
3	Post-hook: `.cs` under `backend/` → `dotnet build <nearest csproj> --artifacts-path .claude/.hookbuild`; `.ts(x)` under `frontend/` → `node node_modules/typescript/bin/tsc -b`; anything else is skipped. Exit 2 with the first 30 errors; logs OK/FAILED with a duration; fails open	`.claude/hooks/post-verify.cs`, a `PostToolUse` entry in `settings.json`, `.claude/hooks/tests/test-post-verify.sh`, `.gitignore` (`.claude/.hookbuild/`)	Desktop authors, CLI verifies	Test script passes, then live P16 HookProbe r shows FAILE then OK in `hooks.log`
4	Both skills	3 files in `.claude/skills/`	Desktop	New session `/skills` list both, plus th P19 question
5	Both agents (`qa-reviewer` tools: Read, Grep, Glob, Bash, plus MCP read tools later)	2 files in `.claude/agents/`	Desktop	`/agents`, th P21 trial run, then `git status` sho the reviewer changed nothing
6	Update CLAUDE.md to match what's verified	`CLAUDE.md`	Desktop, after steps 2–5 pass	Diff review
7	Verify rule loading	—	CLI	P12 output
8	Plugin: Windows python fix, then confirm a hook fires on a `.tsx` edit, then a baseline audit	`docs/evidence/plugin-audit-before.md`	You + CLI	Hook output has no error; audit report saved

#	Correction	Files	Who	Proof
9	MCP, after Stage D1: snapshot	`.mcp.json` `settings.json`	Desktop	`/mcp`

`.mcp.j` Before I hand off the remaining Stage C work to Claude Desktop, please finish your current task safely.

1. Confirm the project directory is `C:\Users\Laxmi Bandaru\claude-code-workshop-assessment`.
2. Inspect `git status` and the current diff.
3. Complete only the current pre-hook task and its related tests if anything remains unfinished.
4. Run the existing pre-hook test suite and report the actual results.
5. Review the staged and unstaged changes. Do not include unrelated documentation changes, generated files, secrets, or databases.
6. Commit only the completed pre-hook work and its relevant tests with a clear commit message, if the changes are ready.
7. Show me the commit hash, `git status`, and the actual test results.

Do not start skills, agents, post-hook, MCP, or other implementation tasks. Do not reset, clean, or discard any changes. After reporting the results, stop so I can hand off the next s'tage to Claude Desktop.

`/c npx` write to `query` SELEC `mcp__e` matche `reviewe`

Stage C cou D1.

To avoid bot session (fini a single bloc

Reply "appr

Show more

Used 2 tools

I could do the read-only checks (1, 2 and 5). I couldn't run the test suite or commit from this Desktop session, so steps 3, 4 and 6 still need your Claude Code CLI session. The suite runs `dotnet run` and .NET isn't installed here. Running `git commit` from here would leave `.lock` files in your `.git` folder.

1. Directory: confirmed, `C:\Users\Laxmi Bandaru\claude-code-workshop-assessment`, on branch `main` at `d222e5c` (baseline).

2. `git status` **and diff:**

File	Change	Who made it	Belongs in the pre-hook commit?
`.claude/hooks/pre-guard.cs`	+21/−5: optional `PRE_GUARD_LOG_FILE` so test runs don't write to the real `hooks.log`	CLI session	Yes
`.claude/hooks/tests/test-pre-guard.sh` (new)	59-case test suite	CLI session	Yes
`docs/evidence/hooks/pre-guard-tests.txt` (new)	Saved results of the suite	CLI session	Yes
`docs/evidence/hooks/pre-guard-live-blocks.log` (new)	2 real blocks recorded while Claude Code was running: `10:16:55 BLOCK Bash rm -rf…` and `10:17:00 BLOCK Write '.env.hookprobe'…`	CLI session	Yes
`.claude/rules/security.md`, `.claude/rules/testing.md`, `CLAUDE.md`, `docs/architecture.md`, `docs/requirements.md`	DB name changed to `expense-management.db`, base URL to 5200	Me (decisions D1 and D5)	No. Separate commit
`docs/evidence/session-01-transcript.txt` (new)	`AssessmentChat.tsx` moved here	Me (decision 3)	No. Separate commit

4. Test results: I didn't run the suite. The only result available is the saved file `docs/evidence/hooks/pre-guard-tests.txt`, which ends with `RESULT: 59 passed, 0 failed`. Treat that as the CLI's earlier run until it's run again.

5. Review: no databases, build output, `.env` files or secrets among the changes. A search for secret-like strings only matched variable names in `pre-guard.cs` (`token`). Nothing is staged.

Finish it in the CLI session

Paste this into the running Claude Code session:

```text
```

```
Finish the pre-hook task only. Do not start skills, agents, post-hook or MCP work,
1. Run: bash .claude/hooks/tests/test-pre-guard.sh  and show the full real output.
2. If all tests pass, overwrite docs/evidence/hooks/pre-guard-tests.txt with this
3. Stage ONLY these paths:
   git add .claude/hooks/pre-guard.cs .claude/hooks/tests/test-pre-guard.sh docs/e
4. Show git diff --cached --stat, then commit:
   git commit -m "Phase 5a: pre-guard test log override, 59-case test suite and li
5. Show the commit hash (git log --oneline -3) and git status. Then stop.
```

After that, `git status` should still list my doc fixes (the five files above) and the transcript move. You can commit those separately:

text

```
git add .claude/rules/security.md .claude/rules/testing.md CLAUDE.md docs/architec
git commit -m "docs: rename database to expense-management.db, base URL 5200, move
```

Then exit the CLI session and tell me. I'll pick up Stage C steps 2–5 from my last plan: the PowerShell matcher, `post-verify.cs` with its test script, both skills and both agents.

7. Stage D — backend implementation
   Work in the existing backend projects:

- `backend/ExpenseManagement.Api`
- `backend/ExpenseManagement.Api.Tests`

Implement or repair the following.
Data layer

- Category, Expense, Budget, and budget-enforcement domain types.
- EF Core `AppDbContext` and Fluent API configuration.
- Decimal precision 18,2.
- `DateOnly` for expense dates.
- Case-insensitive unique category names.
- Unique category/year/month budget constraint.
- Restrict foreign-key delete behaviour.
- EF Core migrations.
- Explicit migration workflow.
- Development-only seed data.
- Isolated in-memory SQLite test databases.

Never use the real development database for tests.
API foundation

- Dependency injection and service layer.
- Thin controllers.
- RFC 7807 ProblemDetails.
- Safe generic 500 responses.
- Validation and consistent error mapping.
- CORS for `http://localhost:5173`.
- OpenAPI JSON and usable API documentation.
- `.http` samples using `http://localhost:5200`.

Categories
Implement or verify:

- `GET /api/categories`
- `GET /api/categories/{id}`
- `POST /api/categories`
- `PUT /api/categories/{id}`
- `DELETE /api/categories/{id}`

Return 409 for duplicate names and for deleting a category that is in use, 404 for missing resources, and 204 for successful deletion.
Expenses
Implement CRUD and list filtering.
Validation:

- Title: 1–100 characters.
- Amount: greater than zero, at most 1,000,000, maximum two decimal places.
- Date: required and not in the future.
- Category: must exist.
- Notes: optional, maximum 500 characters.

Support date range, category, amount range, case-insensitive search, sorting, and paging.
Account for SQLite decimal aggregation and sorting limitations by filtering in SQL and performing the required decimal calculations in C#.
Budgets

- Limit greater than zero.
- Month from 1 to 12.
- Year from 2000 to 2100.
- One budget per category and month.
- Hard budgets reject overspending with HTTP 422.
- Soft budgets accept overspending and return a warning.
- Expense create and update both enforce budget rules.
- Expense updates exclude the existing expense from calculations and correctly handle category/month changes.

Budget status:

- OnTrack: below 80%.
- Warning: 80% through 100%.
- Exceeded: above 100%.

Summaries and reports
Follow the existing requirements contract. Verify the exact query parameters and response shapes in `docs/requirements.md` before implementation.
Implement the documented monthly summary and expense report endpoints, including:

- Monthly totals and counts.
- Category totals and percentages.
- Budget status.
- Empty-month zero values.
- JSON report metadata, date range, totals, category breakdown, and expenses.
- CSV using the agreed header, ISO dates, invariant culture, correct escaping, `text/csv`, and required filename pattern.

Backend tests
Use xUnit to cover successful requests, validation, missing resources, conflicts, budget behaviour, date boundaries, filtering, sorting, paging, summaries, rounding, CSV escaping, and error handling.
For each feature:

1. Implement using the relevant skill and agent.
2. Build and run relevant tests.
3. Use `qa-reviewer` for independent review.
4. Fix verified defects.
5. Record the actual result.
6. Commit at the approved milestone without including unrelated changes.

Show more

Used 15 tools ▸

I've written the whole Stage D backend, but **none of it has been compiled or run yet.** This Desktop session has no .NET SDK, so I couldn't build, test or create the migration. The build-and-fix loop has to happen in your Claude Code CLI session, and that run is also where the real evidence comes from.

Code written (in `backend/`, nothing committed):

- **Data:** the `Category`, `Expense` and `Budget` entities with the Hard/Soft and OnTrack/Warning/Exceeded enums.
  - Money is `decimal(18,2)`, expense dates are `DateOnly`.
  - Category names are unique regardless of case, and there's one budget per category per month.
  - Deleting a category is blocked by the database while it has expenses or budgets.
- **Database:** connection string `Data Source=expense-management.db`. A relative path is resolved against the API project folder, so `dotnet run` and `dotnet ef` use the same file.

- **Seed data:** a seeder that runs only in Development, only on an empty database whose migrations are all applied. It never migrates and never runs in tests.
- **API:** thin controllers that make one service call each, with all rules in the services and every error returned as ProblemDetails.
  - Unexpected errors return a generic 500 with a `traceId` and no exception details.
  - CORS allows only `http://localhost:5173`.
  - OpenAPI JSON plus a Scalar documentation page at `/scalar`.
- **Endpoints:** exactly the FR-01 to FR-16 contract in `docs/requirements.md`.
  - Budgets use `{categoryId, month: "YYYY-MM", limit, type}`.
  - Summaries are `/api/summaries/monthly?year=` and `/api/summaries/categories?month=`.
  - The report is `/api/reports/monthly?month=&format=json|csv`, with CSV header `Date,Title,Category,Amount` and spreadsheet-formula protection.
- **Sample requests:** `ExpenseManagement.Api.http` covers every endpoint, including the 400, 404, 409 and 422 cases, against `http://localhost:5200`.
- **Tests:** 76 test methods with 80 data rows: 17 unit methods / 51 rows and 59 integration methods / 29 rows. Every test is tagged with its FR/BR id.
  - Each integration test gets its own in-memory SQLite database and a fixed "today" of 2026-10-15.
  - A safety check fails the run if a test ever points at a database file.
- **Rules:** no `var` anywhere, and `TreatWarningsAsErrors` is on (NFR-06).

Choices I made that you should know about:

- **Expense lists:** filtering by date, category and text happens in SQL. Amount filtering, sorting and paging happen in C#, because SQLite can't compare or sort decimal columns. That's simpler than the "page in SQL when possible" option in `docs/architecture.md` §5, and I'll update the doc in Stage G.
- **Extra fields:** the category summary rows also include `percentOfTotal`, and the JSON report adds `from`, `to`, `generatedAt` and `expenseCount`. These are additions only; nothing in the contract was removed or renamed.
- **Soft-budget warnings:** you get one when spending moves from OnTrack into Warning, and whenever it's Exceeded. Hard budgets only reject, they never warn.

Two things to watch in your CLI session:

- It has staged the pre-hook files but not committed them yet. My backend files are unstaged, so commit with `git commit` exactly as staged, never `git add -A`.
- `docs/evidence/hooks/pre-guard-live-blocks.log` is excluded by the `*.log` rule in `.gitignore`. Rename it to `.txt` if you want it committed.

What to run in Claude Code CLI

Finish the pending pre-hook commit first. Then start a new session (`/exit`, then `claude`) and paste:

```text
Stage D verification. Backend code under backend/ was authored outside Claude Code
Follow CLAUDE.md and .claude/rules. Do not weaken, skip or delete tests, do not ch
docs/requirements.md, do not use var, and do not touch frontend/ or .claude/.
1. Run: dotnet add backend/ExpenseManagement.Api package Scalar.AspNetCore   (appr
2. Run: dotnet build backend. Fix every compile error and warning (TreatWarningsAs
   Report each fix as file:line, the error, and the fix.
3. Run: dotnet ef migrations add InitialCreate --project backend/ExpenseManagement
4. Run: dotnet test backend. For each failure, decide whether the test or the proc
   docs/requirements.md, fix only the real defect, and re-run. Report the final pa
5. Run: dotnet ef database update --project backend/ExpenseManagement.Api, then st
   dotnet run --project backend/ExpenseManagement.Api --launch-profile http and cc
   GET /api/categories (6 seeded rows), GET /api/summaries/categories?month=<curre
   Stop the API afterwards.
6. Show git status. Do not commit. Report real output only.
```

When it passes, commit only the backend:

```text
git add backend/
git commit -m "Phase 7-9: data layer, InitialCreate migration, API and backend tes
```

Still open for Stage D (needs Stage C first): the `api-implementer` and `qa-reviewer` agents and both skills don't exist yet. So the "implement with the skill and agent, then review" loop can't be shown for this code. Until a qa-reviewer pass runs, `docs/claude-code-evidence.md` will record the backend as Desktop-authored, build-verified in the CLI, and not yet independently reviewed. Once the build is green, I can write the Stage C skills and agents so you can run a real qa-reviewer pass on this backend.

Please inspect the `.claude` folder in my current assessment project:
`C:\Users\Laxmi Bandaru\claude-code-workshop-assessment`
I noticed that the required skills and subagent files have not been created yet, even though I previously asked you to create them.
Please complete the missing work as follows:

1. Inspect the existing `.claude` folder and identify which skill and subagent folders or files are missing. Do not recreate files that already exist and are correct.
2. Create the missing skill folders and files:
   - `.claude/skills/aspnet-endpoint-workflow/SKILL.md`

- `.claude/skills/aspnet-endpoint-workflow/checklist.md`
- `.claude/skills/expense-reporting/SKILL.md`

3. Create the missing subagent files:
   - `.claude/agents/api-implementer.md`
   - `.claude/agents/qa-reviewer.md`
4. Follow the existing `CLAUDE.md`, rules, project architecture, and `docs/requirements.md`. Make the skill instructions practical and reusable for the Expense Management API project.
5. Define `api-implementer` as a backend implementation specialist responsible for following the established architecture, implementing changes, and running relevant tests.
6. Define `qa-reviewer` as an independent code reviewer. It must be read-only and must not have Edit or Write tools. It should review correctness, security, validation, test coverage, and compliance with the requirements without modifying files.
7. Before creating or editing anything, inspect the relevant existing files and preserve all unrelated work. Do not overwrite correct content or change the API contract.
8. After creating the missing files, show me the exact file paths created or updated and briefly explain the purpose of each file.
9. Validate the Markdown structure and required frontmatter or configuration supported by the installed Claude Code version. Do not assume that creating a file proves the skill or subagent works.
10. Clearly distinguish files created in Claude Desktop from features actually tested in Claude Code CLI. Do not claim the skills or subagents have loaded or executed unless you verify that using the real Claude Code CLI runtime.

Important constraints:

- Work only in the assessment project folder.
- The separate demo project at `C:\Users\Laxmi Bandaru\claude-code-workshop` is strictly read-only. Do not modify files or run commands inside it.
- Do not reset, clean, or discard Git changes.
- Do not modify unrelated hooks, plugin settings, MCP configuration, backend code, or frontend code as part of this task.
- If you cannot access or edit the project files from this Claude Desktop session, tell me exactly what is blocking you instead of claiming the work is complete.

Please create the missing skills and subagent files now, then report the results and any remaining Claude Code CLI verification steps.

Show more

Ran a command on your computer ▸

The audit confirms `.claude/skills/` and `.claude/agents/` don't exist. Nothing in them needs preserving, and I won't touch the hooks, settings, backend or docs. Creating the five files now.

Used 2 tools ▸

The five files now exist on disk in your assessment folder. I created them from this Desktop session and checked their structure here. Nothing shows yet that Claude Code has loaded or run them; that check has to happen in the CLI.

`.claude/skills/` and `.claude/agents/` didn't exist before, so there was nothing to overwrite. I didn't touch the hooks, settings, MCP, plugin, backend, frontend or docs. Your CLI session's staged pre-hook changes are still staged and uncommitted.

File (created)	Purpose
`.claude/skills/aspnet-endpoint-workflow/SKILL.md`	The reusable skill: an 8-step workflow for any ASP.NET Core + EF Core endpoint (contract → validator → service → thin controller → tests → `.http` sample → docs → real build and test). It contains no expense, budget or category wording, so it works in other projects.
`.claude/skills/aspnet-endpoint-workflow/checklist.md`	A done-checklist that `SKILL.md` links to (contract, validation, controller, data access, tests, delivery).
`.claude/skills/expense-reporting/SKILL.md`	The project-specific skill. It encodes the reporting rules from FR-10 to FR-16 and BR-01 to BR-08, and maps each one to its backend file. It covers: month boundaries, decimal and rounding rules, the SQLite "sum and compare in C#" limit, budget status and Hard/Soft logic, summary and report response shapes, CSV format and escaping, the MCP cross-check SQL, and the list of required tests. If the skill and `docs/requirements.md` disagree, the requirements win.
`.claude/agents/api-implementer.md`	The backend implementer. Tools: Read, Edit, Write, Bash, Grep, Glob. It may edit only `backend/`, must use both skills, explicit types, and migrations through `dotnet ef`, must never change the API contract, and has to report real build and test output.
`.claude/agents/qa-reviewer.md`	The independent reviewer. Tools: Read, Grep, Glob, Bash, with no Edit or Write. It reviews requirements, correctness, validation, security, rules and tests, cross-checks totals through MCP if that's connected, and must show that `git status` is the same before and after.

What I checked here:

- All 4 frontmatter blocks parse as valid YAML.

- Each `name` matches its folder or file name, and each `description` says when to use it.
- No trailing whitespace (the defect found in the demo project).
- `qa-reviewer`'s `tools` list contains no Edit or Write tool.
- The reusable skill contains no domain words.

Two limits you can mention to the assessor:

- The reviewer still has Bash, so its read-only behaviour is enforced by its instructions, plus the pre-guard hook blocking destructive commands. Its tool list alone doesn't guarantee it.
- I didn't add the subagent `skills:` preload field, because I can't confirm from here that your installed version supports it. The agents point to the skills in their instructions instead.

Checking it in Claude Code CLI

Exit the current session and start a new one, because skills and agents load at startup.

1. Run `/skills`. Both `aspnet-endpoint-workflow` and `expense-reporting` should be listed.
2. Run `/agents`. Both agents should be listed with the tools shown above.
3. Paste:

```text
Which skill would you use to (a) add a GET /api/categories endpoint and (b) bui
Answer from the skill descriptions only, and explain why one is reusable and or
```

Expect (a) `aspnet-endpoint-workflow` and (b) `expense-reporting`.
4. Once the backend from Stage D builds, run a real subagent test:

```text
Run git status --short and save the output. Then use the qa-reviewer subagent t
(FR-01 to FR-04) against docs/requirements.md and .claude/rules, running dotnet
After it finishes, run git status --short again and confirm the reviewer change
Finally answer: can qa-reviewer edit files? Answer from its configuration.
```

5. If both lists show up correctly, commit only these files:

```text
```

```
git add .claude/skills .claude/agents
git commit -m "Phase 6: aspnet-endpoint-workflow and expense-reporting skills,
```

Paste the output of steps 1–4 here and I'll record it in `docs/claude-code-evidence.md`, marking anything that didn't show up as NOT DEMONSTRATED.

8. Stage E — frontend implementation
   Work in frontend/.
   Implement or repair:

- Strict TypeScript DTOs matching the actual API contract.
- Typed API client and structured ApiError parsing for ProblemDetails.
- API modules for categories, expenses, budgets, summaries, and reports.
- Reusable loading, empty, error, and confirmation components.
- Navigation for Expenses, Categories, Budgets, and Summary & Reports.
- Vite /api proxy to http://localhost:5200.

Show more

Expenses page
Provide paged results, filters, add/edit forms, validation, delete confirmation, field-level errors, 422 hard-budget errors, soft-budget warnings, and loading/empty/error states.
Categories page
Provide list, add, rename, and delete functionality, including useful conflict messages.
Budgets page
Provide month selection, budget CRUD, Hard/Soft selection, status indicators, progress bars, and accessible text labels.
Summary and Reports page
Provide monthly totals, category percentages, budget status, JSON report viewing, and CSV download.
Frontend tests
Use Vitest and React Testing Library. Cover validation, expense creation, budget errors and warnings, filter parameters, empty/error states, summary rendering, and CSV download.
Run npm test and npm run build. Report actual results, and distinguish mocked frontend tests from live integration tests.

Same approach as the backend, but this time I can verify it: npm works in my cloud workspace. I'll copy your current frontend config and source there, build the frontend, run its tests and build, then write the finished files back into `frontend/`. First, staging the existing files so I work from your real versions.

Ran 2 commands, used a tool ▸

I confirmed strict mode is on by default in TypeScript 6 (an implicit `any` and an unsafe `undefined` assignment both failed), so I'll leave the tsconfig alone. Writing the types and the API layer.

Ran 12 commands, used 2 tools ▸

The Stage E frontend is finished and in your `frontend/src/` folder (40 files). Tests and build pass with mocked API responses; it has not been tested against the live API yet. That needs the backend from Stage D to build first.

Real results (run in my cloud workspace on a fresh `npm ci` of your lockfile, on Linux rather than your Windows machine):

Check	Result
`npx vitest run`	**7 test files, 49 tests passed, 0 failed,** with no `act()` or console warnings
`npm run build` (`tsc -b` + `vite build`)	Passed: 41 modules, JS bundle 249.82 kB (75.83 kB gzip)
`npx oxlint src`	0 warnings (exit 0)
Rule checks	No `any`. `fetch` is called only in `src/api/client.ts`. No new npm packages

I checked that the copies in `frontend/` are identical to what I tested. `main.tsx`, `vite.config.ts` (already has the `/api` → `http://localhost:5200` proxy), the tsconfigs and `package.json` are unchanged. TypeScript 6 already applies strict mode by default: an implicit `any` fails to compile.

What's in it:

- **Types and API layer:** `types/api.ts` mirrors the backend contracts.
  - `api/client.ts` turns any error body (RFC 7807 problem details) into a typed `ApiError` with status, title, detail, per-field errors and a flag for a hard-budget 422.
  - It uses `VITE_API_URL` if set; otherwise relative `/api` paths go through the Vite proxy.
  - There's one API module each for categories, expenses, budgets, summaries and reports.
- **Shared components:** loading, empty, error and success messages, field errors linked to their inputs with `aria-describedby`, a two-step Delete → Confirm button, a pager, a category picker, and a status badge plus progress bar. Budget status is always shown as text ("Warning") as well as colour.
- **Pages** (simple buttons switch between them):

  - **Expenses:** paged table, filters (search, category, date range, amount range, sort) applied when you submit, an add/edit form whose client checks mirror the backend limits, server field errors under the right input, a 422 "Hard budget exceeded…" message, soft-budget warnings after a save, and loading/empty/error states.
  - **Categories:** list, add, inline rename, delete. A duplicate name shows the 409 message under the field; deleting a category that still has expenses shows why it wasn't deleted.
  - **Budgets:** month picker, add/edit/delete, Hard/Soft choice. Progress bars carry `aria-valuetext` and are capped at 100% visually while the text shows the real percentage.
  - **Summary & Reports:** category summary (totals, share of month, budget status), monthly totals for the year, the JSON report rendered as a table, and a CSV download that uses the file name the server sends.
- **Tests:** client error parsing, validation boundaries, and all four pages (validation, create, 422, soft warnings, filter query string, empty/error states, summary rendering, CSV download), plus navigation.

Mocked vs live: every frontend test replaces `fetch` with an in-memory fake (`src/test/fetchMock.ts`), so no backend ran. These tests prove the UI logic against the documented contract, not against the real API. The live check (comparing the frontend types with `/openapi/v1.json`, clicking through the pages, and checking data survives a restart) still has to happen.

Still to do in Claude Code CLI

1. **Re-run on Windows** (your `node_modules` has Windows binaries):

```text
In frontend/: run npm test and npm run build and show the real summary lines. D
```

2. **Cognine plugin audits (P41):** run the three `/react-coding-standards-best-practices:…` commands on `frontend/src` and save the results as `docs/evidence/plugin-audit-before.md`, fix the Mandatory/High findings, then re-run them for `plugin-audit-after.md`. The plugin's own Python hooks will keep failing until the `python3` Windows setting is fixed.
3. **Commit the frontend on its own:**

```text
git add frontend/src
git commit -m "Phase 10: React + TypeScript frontend with mocked-API tests"
```

4. **Stage F, once the backend is green:** run the M1 alignment prompt from the playbook: compare the frontend types with the live OpenAPI document, click through the UI, and check data survives a restart.

```
$

Laxmi Bandaru@COGNINE-L220 MINGW64 ~/claude-code-workshop-
assessment (main)
$ dotnet add
backend/ExpenseManagement.Api/ExpenseManagement.Api.csproj package
Microsoft.AspNetCore.OpenApi --version 10.0.11
info : X.509 certificate chain validation will use the default trust store selected
by .NET for code signing.
info : X.509 certificate chain validation will use the default trust store selected
by .NET for timestamping.
info : Adding PackageReference for package 'Microsoft.AspNetCore.OpenApi'
into project
'backend/ExpenseManagement.Api/ExpenseManagement.Api.csproj'.
info : Restoring packages for C:\Users\Laxmi Bandaru\claude-code-workshop-
assessment\backend\ExpenseManagement.Api\ExpenseManagement.Api.cspr
oj...
info :   GET https://api.nuget.org/v3-
flatcontainer/microsoft.aspnetcore.openapi/index.json
info :   OK https://api.nuget.org/v3-
flatcontainer/microsoft.aspnetcore.openapi/index.json 279ms
info :   GET https://api.nuget.org/v3-
flatcontainer/microsoft.aspnetcore.openapi/10.0.11/microsoft.aspnetcore.opena
pi.10.0.11.nupkg
info :   OK https://api.nuget.org/v3-
flatcontainer/microsoft.aspnetcore.openapi/10.0.11/microsoft.aspnetcore.opena
pi.10.0.11.nupkg 12ms
info :   GET https://api.nuget.org/v3-flatcontainer/microsoft.openapi/index.json
info :   OK https://api.nuget.org/v3-flatcontainer/microsoft.openapi/index.json
233ms
info :   GET https://api.nuget.org/v3-
flatcontainer/microsoft.openapi/2.7.5/microsoft.openapi.2.7.5.nupkg
info :   OK https://api.nuget.org/v3-
flatcontainer/microsoft.openapi/2.7.5/microsoft.openapi.2.7.5.nupkg 19ms
info : Installed Microsoft.AspNetCore.OpenApi 10.0.11 from
https://api.nuget.org/v3/index.json to C:\Users\Laxmi
Bandaru\.nuget\packages\microsoft.aspnetcore.openapi\10.0.11 with content
hash
R/1EATnPLU+gRfB6lwVkMcymmyAY5ppBBdRN/5lhNEiT3xP1sWccuSFkU/f1lQv
N/WgRq5Vn8AhCdE3fqgsL/w==.
info : Installed Microsoft.OpenApi 2.7.5 from https://api.nuget.org/v3/index.json
```

to C:\Users\Laxmi Bandaru\.nuget\packages\microsoft.openapi\2.7.5 with content hash 0FA67RSnRM4tcBKqiqVu/HPdZ9+QOKbmeRjxRUGTCjPU4C0bmUhd97Dso7Yild5P7nOV6GxJ2xrK0Kv/O9xp0w==.
info : CACHE https://api.nuget.org/v3/vulnerabilities/index.json
info : CACHE https://api.nuget.org/v3-vulnerabilities/2026.10.19.23.33.13/vulnerability.base.json
info : CACHE https://api.nuget.org/v3-vulnerabilities/2026.10.19.23.33.13/2026.10.26.05.34.09/vulnerability.update.json
info : Package 'Microsoft.AspNetCore.OpenApi' is compatible with all the specified frameworks inproject 'backend/ExpenseManagement.Api/ExpenseManagement.Api.csproj'.
info : PackageReference for package 'Microsoft.AspNetCore.OpenApi' version '10.0.11' updated in file 'C:\Users\Laxmi Bandaru\claude-code-workshop-assessment\backend\ExpenseManagement.Api\ExpenseManagement.Api.csproj'.
info : Generating MSBuild file C:\Users\Laxmi Bandaru\claude-code-workshop-assessment\backend\ExpenseManagement.Api\obj\ExpenseManagement.Api.csproj.nuget.g.targets.
info : Writing assets file to disk. Path: C:\Users\Laxmi Bandaru\claude-code-workshop-assessment\backend\ExpenseManagement.Api\obj\project.assets.json
log : Restored C:\Users\Laxmi Bandaru\claude-code-workshop-assessment\backend\ExpenseManagement.Api\ExpenseManagement.Api.csproj (in 2.5 sec).

Laxmi Bandaru@COGNINE-L220 MINGW64 ~/claude-code-workshop-assessment (main)
$ dotnet restore backend/ExpenseManagement.Api/ExpenseManagement.Api.csproj
Restore complete (0.9s)

Build succeeded in 2.1s

Laxmi Bandaru@COGNINE-L220 MINGW64 ~/claude-code-workshop-assessment (main)
$ dotnet build backend/ExpenseManagement.Api/ExpenseManagement.Api.csproj
Restore complete (0.7s)
ExpenseManagement.Api net10.0 failed with 1 error(s) (8.2s)
C:\Users\Laxmi Bandaru\claude-code-workshop-assessment\backend\ExpenseManagement.Api\Program.cs(10,7): error CS0246: The type or namespace name 'Scalar' could not be found (are you missing a using directive or an assembly reference?)

Build failed with 1 error(s) in 10.2s....build is failing ..solve this

Show more

The build fails only because the `Scalar.AspNetCore` package was never added. `Program.cs` uses it for the API docs page at `/scalar`, and the command you ran added `Microsoft.AspNetCore.OpenApi` instead (updating it to 10.0.11 is fine and can stay). Only one error was reported, so the rest of the backend compiled.

Run this in the same Git Bash window:

```bash
dotnet add backend/ExpenseManagement.Api/ExpenseManagement.Api.csproj package Scal
dotnet build backend
```

The first command adds the latest Scalar release that's compatible with .NET 10. You approved this package earlier (decision 1). Then run `dotnet build backend` for the whole solution rather than just the API project, so the test project compiles too.

If `dotnet add` fails or you'd rather not add the package, tell me and I'll take the docs UI out instead. That means deleting two lines in `Program.cs`: `using Scalar.AspNetCore;` and `app.MapScalarApiReference();`. You'd still have `/openapi/v1.json` and the `.http` file for API documentation.

If the build now shows new errors (for example from `TreatWarningsAsErrors` or in the test project), paste the full error list and I'll fix them. Once it builds, carry on with the rest of the Stage D prompt in the CLI:

```bash
dotnet ef migrations add InitialCreate --project backend/ExpenseManagement.Api
dotnet test backend
dotnet ef database update --project backend/ExpenseManagement.Api
```

> how to run both frontend and backend simulatenously in visual studio code...i think we should use 2 terminals..give me the commands

Yes, two terminals: one for the backend, one for the frontend. Start the backend first, because the frontend sends its `/api` calls to it on port 5200.

Open the two terminals in VS Code

1. Open the folder: **File → Open Folder →** `C:\Users\Laxmi Bandaru\claude-code-workshop-assessment`.
2. Open a terminal with **Ctrl** + `` ` `` (the backtick key). If you want Git Bash, pick it from the ∨ menu next to the +.

3. Open the second terminal with the + button, or use **split** (Ctrl+Shift+5) to see both side by side.

Terminal 1: backend (API on http://localhost:5200)

The first time only, create the database:

```bash
dotnet ef database update --project backend/ExpenseManagement.Api
```

Then start the API:

```bash
dotnet run --project backend/ExpenseManagement.Api --launch-profile http
```

Wait for `Now listening on: http://localhost:5200`. Leave this terminal running.

Terminal 2: frontend (React on http://localhost:5173)

```bash
cd frontend
npm install
npm run dev
```

You only need `npm install` the first time, or after `package.json` changes. Wait for `Local: http://localhost:5173/`.

Open these in your browser

URL	What it is
http://localhost:5173	The React app
http://localhost:5200/scalar	API documentation page
http://localhost:5200/openapi/v1.json	OpenAPI document
http://localhost:5200/api/categories	Quick API check (should show 6 seeded categories)

To stop: click into each terminal and press Ctrl + C.

If something goes wrong

- **The app loads but shows "The API could not be reached" or 404s:** Terminal 1 isn't running, or it's on the wrong port. Use `--launch-profile http`, because the `https` profile uses port 5050.

- **The API returns 500 errors, or you see "Skipping development seed data" in Terminal 1:** the database wasn't migrated. Run `dotnet ef database update --project backend/ExpenseManagement.Api`, then restart Terminal 1.
- **The backend build says files are locked:** stop the running API with Ctrl + C before running `dotnet build` or `dotnet test`.

Task: Transform the Expense Management Frontend into a Premium, Modern UI
Act as a senior frontend engineer, UI/UX designer, and React + TypeScript specialist.
I already have an Expense Management application running at `http://localhost:5173`. The current interface is functional but looks basic and outdated. I want you to transform it into a professional, premium-quality expense management dashboard with beautiful styling, smooth animations, excellent usability, and a fully responsive layout.
1. First, inspect the existing project
Work only in the current assessment repository:
`C:\Users\Laxmi Bandaru\claude-code-workshop-assessment`
Before making changes:

- Inspect `frontend/`, its `package.json`, existing React components, styles, API layer, DTOs, and tests.
- Read `CLAUDE.md`, the applicable `.claude/rules/`, and `docs/requirements.md`.
- Understand the existing API endpoints and response types.
- Identify the existing frontend framework, styling approach, and installed dependencies.
- Preserve the working backend, API contracts, database, migrations, hooks, MCP configuration, and existing Git changes.

Do not assume the API supports endpoints or response properties that you have not verified.
2. Design direction
Create a premium personal finance dashboard inspired by modern fintech products such as Linear, Stripe Dashboard, and contemporary SaaS analytics platforms.
Use a consistent visual design system:

- Soft neutral background with white or subtly tinted surfaces.
- Deep navy or charcoal text.
- A refined indigo/blue primary accent.
- Green for healthy budget status, amber for warnings, and red for exceeded budgets or errors.
- Subtle borders, elegant shadows, consistent rounded corners, and generous spacing.
- A modern font such as Inter, using an existing font or a reliable fallback.

- Clear typography hierarchy and carefully aligned layouts.
- Consistent spacing, buttons, inputs, cards, badges, tables, and dialogs.
- Light theme as the default, with a dark theme only if it can be implemented cleanly.

Avoid excessive gradients, oversized empty areas, unnecessary decoration, and generic-looking components. Prioritise readability and a professional financial-product appearance.

3. Application layout

Replace the basic horizontal navigation with a polished application shell.

Sidebar

Create a collapsible left sidebar containing:

- Expense Management logo and application name.
- Overview.
- Expenses.
- Categories.
- Budgets.
- Summary & Reports.
- A small profile or application-information section at the bottom.

Highlight the active navigation item and include appropriate icons using an existing icon library or a lightweight compatible dependency.

On mobile, convert the sidebar into a usable drawer with a menu button.

Top header

Include:

- Current page title and short contextual description.
- Breadcrumbs where useful.
- Current month or selected reporting period.
- A primary action such as “Add expense” where appropriate.

Do not add fake notifications, fake user accounts, or decorative controls that do nothing.

4. Overview dashboard

Create an attractive overview page using data from the existing API wherever supported.

Include summary cards for:

- Total spending for the selected month.
- Number of expenses.
- Average expense, calculated from verified data.
- Budget utilisation or remaining budget, if the API provides enough information.

Add:

- A monthly spending trend chart if the API provides suitable historical data.
- Category spending breakdown.
- Budget status overview.
- Recent expenses table.

- Quick action to add an expense.

Use an existing charting library if one is already installed. If a new dependency is necessary, explain why before adding it.
All calculations must use actual API data. Do not fabricate chart values, totals, or statistics. Show a useful empty state when data is unavailable.
5. Expenses page
Redesign the existing Expenses page shown in my screenshot.
Expense list
Create a professional data table with:

- Date.
- Expense title.
- Category badge.
- Amount, aligned to the right.
- Actions for edit and delete.

Add polished hover states, clear row separators, responsive handling, and a readable amount format.
If pagination is supported by the existing API, implement it correctly.
Search and filters
Organise the filters into a compact toolbar or collapsible filter panel:

- Search by title or notes.
- Category.
- Start date and end date.
- Minimum and maximum amount.
- Sort field.
- Sort direction.

Provide clear Apply and Reset actions. Make sure the filters use the actual backend query parameter names.
Add and edit expense
Create a beautiful form, preferably in a right-side drawer or modal on desktop and a mobile-friendly full-screen or bottom-sheet layout on smaller screens.
Include:

- Expense title.
- Amount.
- Date.
- Category selection.
- Optional notes.

Requirements:

- Label every field.
- Display validation errors next to the relevant fields.
- Show loading states while saving.
- Prevent duplicate submissions.
- Preserve user input when appropriate after validation failures.
- Show a success toast after a successful operation.

- Display useful API error messages.
- Close the form only after a successful save or an explicit cancel action.

Delete confirmation
Use a confirmation dialog instead of deleting immediately.
Explain what will happen and display the expense title where available. Show a success or error notification after the operation.
6. Categories page
Create a clean category management interface with:

- Category cards or a well-designed table.
- Category name.
- Add category action.
- Edit or rename action.
- Delete action with confirmation.
- Loading, empty, success, and error states.

Handle duplicate-category conflicts and categories that cannot be deleted because they are in use.
Do not invent category icons, descriptions, or usage counts unless they are available or can be accurately derived from existing API data.
7. Budgets page
Build a visually engaging monthly budget management page.
Include:

- Month selector.
- Budget creation and editing form.
- Category selection.
- Budget amount.
- Hard and Soft enforcement selection.
- Current spending and remaining budget, where available.
- Progress bars.
- Status badges.

Use the project's exact budget rules:

- OnTrack: spending below 80%.
- Warning: spending from 80% through 100%.
- Exceeded: spending above 100%.
- Hard budgets reject overspending with HTTP 422.
- Soft budgets permit overspending and may return warnings.

Use accessible status labels in addition to colours. Handle 422 responses and soft-budget warnings explicitly. Do not duplicate budget calculations incorrectly in the frontend.
8. Summary & Reports page
Create an analytics-style reporting interface with:

- Reporting month selector.
- Monthly totals and expense counts.
- Category breakdown with percentages.

- Budget status overview.
- Monthly expense report in JSON format, if supported by the existing API.
- Download CSV action.

Use clear chart legends, readable labels, useful tooltips, and responsive chart containers.
The CSV download must use the existing report endpoint and preserve the server-generated CSV format. Show a clear error state if the report cannot be downloaded.
9. Animations and micro-interactions
Add smooth, restrained animations throughout the application:

- Fade-in and slight upward movement when pages load.
- Subtle card hover elevation.
- Smooth sidebar expansion and collapse.
- Modal and drawer entrance/exit transitions.
- Button hover and pressed states.
- Animated loading indicators or skeleton placeholders.
- Smooth progress-bar transitions.
- Toast notifications with appropriate transitions.
- Clear focus states for keyboard navigation.

Keep animations short and purposeful. Avoid excessive bouncing, spinning, parallax, or distracting movement.
Prefer CSS transitions and keyframes. If Framer Motion or another animation library is already installed, reuse it where appropriate. Do not add a new animation dependency without explaining the benefit.
Respect `prefers-reduced-motion` so users who disable motion are not forced to experience animations.
10. Frontend engineering requirements
Maintain the existing React + TypeScript architecture.

- Use functional React components and appropriate hooks.
- Use strict TypeScript; do not introduce `any`.
- Define reusable typed components for buttons, inputs, dialogs, badges, cards, tables, and notifications where appropriate.
- Keep HTTP requests inside the existing API layer.
- Reuse existing DTOs and types rather than duplicating API contracts.
- Handle loading, empty, success, validation, and server-error states.
- Prevent unnecessary API requests and avoid stale state.
- Keep components reasonably small and maintainable.
- Use semantic HTML and accessible form labels.
- Support keyboard navigation and visible focus indicators.
- Ensure sufficient colour contrast.
- Make the interface responsive across desktop, tablet, and mobile screens.
- Avoid introducing a state-management library unless genuinely needed.
- Do not replace working business logic just to simplify the UI.

Use the existing styling system if it is suitable. If the project already uses Tailwind CSS, continue using it. Otherwise, choose a consistent approach that fits the existing project rather than mixing multiple styling systems unnecessarily.

11. API integration is mandatory

This is not a static UI mockup.

Every existing feature must continue working with the real backend:

- Expense CRUD.
- Expense filtering, sorting, and paging.
- Category CRUD.
- Budget management and enforcement.
- Monthly summaries.
- Category summaries.
- JSON and CSV reports.

Use the existing API base URL and Vite proxy configuration. Verify the actual routes and DTOs before integrating each page.

Do not introduce hardcoded production data or fake successful responses. If the backend is unavailable, show a clear error or empty state rather than pretending that requests succeeded.

12. Testing and verification

After implementation:

1. Inspect the changes and identify the files modified.
2. Run the existing frontend tests.
3. Add or update React Testing Library and Vitest tests for any changed behaviour.
4. Run `npm test`.
5. Run `npm run build`.
6. Fix genuine TypeScript, build, accessibility, and test failures.
7. Verify that navigation, forms, filters, confirmation dialogs, and report downloads still work.
8. If the backend is running, perform a live integration smoke test.
9. If browser automation is available, inspect the rendered pages at desktop and mobile widths.
10. Report the actual commands run, their results, and any checks that could not be performed.

Do not weaken tests, remove functionality, change the backend API contract, or claim that a check passed if it was not executed.

13. Git and scope safety

- Work only inside `frontend/`, except when a necessary project-level change is explicitly explained and approved.
- Do not edit backend code, migrations, database files, `.claude/` configuration, hooks, MCP settings, or plugin settings.
- Preserve existing uncommitted work.
- Inspect `git status` before editing.

- Never run `git reset --hard`, `git clean`, force-push, or other destructive Git commands.
- Do not stage or commit changes unless I explicitly request it.
- Avoid unnecessary dependencies and large rewrites.

14. Implementation approach
    Work incrementally:

    1. Inspect the existing frontend and present a concise redesign plan.
    2. Identify the existing components and styles that can be reused.
    3. Implement the application shell and design system.
    4. Redesign Expenses.
    5. Redesign Categories and Budgets.
    6. Redesign Summary & Reports.
    7. Add responsive layouts, animations, accessibility, and polished feedback states.
    8. Run tests and the production build.
    9. Fix issues and report the final results.

You may continue implementing after the initial inspection if the requirements and architecture are clear. Do not stop after producing a plan unless a significant decision or dependency requires my approval.
Final goal: Deliver a polished, premium-quality, animated, fully responsive React + TypeScript expense management application that looks like a production fintech product while preserving all existing backend integration and business

Show more

Ran 8 commands, loaded a skill, used a tool ▸

The only errors are in the old tests, which no longer match the new page props. Lint is clean. Rewriting the test suite for the new UI, keeping every earlier scenario and adding the new behaviour.

Ran a command ▸

Ran 13 commands, read 10 files ▸ +8

Only the 40 UI files changed. The API layer, hooks, DTO types and validation are byte-for-byte unchanged, so the backend contract is untouched. Copying the 40 files into your `frontend/src`:

Ran 4 commands, shared files, used a tool ▸

I've finished the redesign and tested it in my cloud workspace, but it isn't in your project yet: your computer dropped off the connection just before I could copy the 40 changed files in. The zip above has them all.

What's new:

- **App shell:** a collapsible sidebar with all five sections (Overview, Expenses, Categories, Budgets, Summary & Reports), active-page highlight and icons. It becomes a slide-out drawer with a menu button on phones. Every page has a header with breadcrumbs, a description, a month picker where it matters, and its main action.
- **Overview:** four summary cards (total spent, number of expenses, average expense, budget used), a monthly trend chart with hover tooltips and a table view, a category breakdown, recent expenses, and budget status. Every figure comes from the real API; empty months show empty states rather than invented numbers.
- **Expenses:** search plus a collapsible filter panel (it uses the backend's real query parameter names), a cleaner table with category badges and right-aligned amounts, and paging.
  - Add and edit open in a side drawer (a full-screen sheet on phones). It shows field errors and the hard-budget 422 message in place, keeps your input, and blocks double-submits.
  - Soft-budget warnings and successes appear as toast notifications.
  - Delete asks for confirmation in a dialog first.
- **Categories, Budgets, Summary & Reports:**
  - Categories are cards. Add and rename happen in a drawer, and you get clear messages when a name is a duplicate or a category still has expenses.
  - Budget cards show progress bars plus a text status label, and explain Hard vs Soft.
  - The reports page has charts, a category table, the JSON report view and the CSV download.
- **Motion and accessibility:** short CSS animations everywhere, turned off for users who choose reduced motion. Keyboard focus is visible and trapped inside open dialogs and drawers, and fields have proper labels.
- **No new npm packages:** icons and charts are hand-written SVG. The API layer, data types, hooks and validation are unchanged, so the backend contract is exactly as before.

Verification (my cloud workspace, Linux):

Check	Result
`npx vitest run`	**8 test files, 66 tests passed**, no warnings. All the earlier scenarios are still covered, plus new ones for the Overview figures, chart tooltip and table view, Escape-to-close, cancelling a delete, delete failures, budget edit/delete and navigation
`npm run build`	Passed
`npx oxlint src`	0 warnings
Browser screenshots	Desktop (1440px) and phone (390px), with no console errors. These ran against a **sample mock API** on port 5200, so they show layout only, not real data

The screenshots caught two bugs the tests couldn't, and both are fixed: the filter panel showed even when collapsed, and dialogs were positioned incorrectly.

Not done yet: a live check against your real backend and database, and a run on Windows. I also didn't build a dark theme or load the Inter web font (it falls back to your system font).

To apply it:

1. Unzip over `frontend/` so the `src` folders merge. Or tell me once your computer is back online and I'll copy the files in directly.
2. In Git Bash, remove the one file the redesign replaces: `git rm frontend/src/components/ConfirmButton.tsx`.
3. In `frontend/`, run `npm test` and `npm run build`. Then start the backend and `npm run dev`, and click through every page against your real data.
4. Commit when you're happy: `git add frontend/src` and `git commit -m "Frontend: premium UI redesign"`.